
pyTempico

Release 2.0.4

Tausand Electronics

Jun 26, 2026

CONTENTS:

1	pyTempico package	3
1.1	TempicoDevicesSearch class	3
1.2	TempicoDevice class	4
1.3	TempicoChannel class	27
2	Examples	35
2.1	Settings examples	35
2.1.1	readIdentifiersExample.py	35
2.1.2	readSettingsExample.py	36
2.1.3	writeSettingsExample.py	37
2.1.4	dateTimeExample.py	39
2.2	Measurement examples	42
2.2.1	singleMeasureExample.py	42
2.2.2	modeTwoMeasureExample.py	43
2.2.3	numberOfRunsMeasureExample.py	44
2.2.4	numberOfStopsMeasureExample.py	45
2.2.5	multipleMeasureSingleChExample.py	47
2.3	TP1200 examples	50
2.3.1	calibrateExample.py	50
2.3.2	calibrateOffsetExample.py	51
2.4	Advanced examples	57
2.4.1	fcsExample.py	57
2.4.2	multipleTauCorrelator.py	63
2.4.3	g2Example.py	68
2.4.4	g2Algorithm.py	81
3	Indices and tables	85
	Python Module Index	87
	Index	89



PyTempico is a Python library developed by Tausand Electronics, with Joan Amaya and David Guzman as the lead developers. It enables seamless interaction with Tempico, Tausand's Time-to-Digital Converter devices, facilitating data communication and control. The library relies on key modules such as hid and pyserial for efficient device connectivity. For more information, visit tausand.com

Release 2.0.4

PYTEMPICO PACKAGE

Created on Jan 30 2024

@author: David Guzman at Tausand Electronics
dguzman@tausand.com
<https://www.tausand.com>

Core class and methods for PyTempico library.

To use with Tausand Electronics' Time-to-Digital Converters (TDCs) of the family *Tausand Tempico*.

1.1 TempicoDevicesSearch class

class `pyTempico.core.TempicoDevicesSearch`

A class for discovering Tempico devices.

This class provides methods to search for Tempico devices in a network or connected system.

findDevices ()

Finds and verifies whether a device with the given VID and PID is a Tempico device.

This function takes the Vendor ID (VID) and Product ID (PID) as inputs, converts them to integers, and attempts to open the device using these values. It then checks if the manufacturer and product strings match the expected values for a Tempico device.

Parameters

- **vid_s** (*str*) – The Vendor ID (VID) of the device in string format.
- **pid_s** (*str*) – The Product ID (PID) of the device in string format.

Returns *True* if the device is a Tempico, *False* otherwise.

Return type `bool`

getVidPid (*vid_pid_information*)

Extracts the Vendor ID (VID) and Product ID (PID) from a string and returns them as a tuple.

This function processes a string that contains the VID and PID information in the format 'VID:PID=xxxx:yyyy'. It splits the string and retrieves the VID and PID values, returning them as a tuple of strings.

Parameters **vid_pid_information** (*str*) – A string containing the VID and PID information.

Returns A tuple containing the VID and PID as strings (vid, pid).

Return type tuple

tryOpenDevices (vid_s, pid_s)

Finds and verifies whether a device with the given VID and PID is a Tempico device.

This function takes the Vendor ID (VID) and Product ID (PID) as inputs, converts them to integers, and attempts to open the device using these values. It then checks if the manufacturer and product strings match the expected values for a Tempico device.

Parameters

- **vid_s** (*str*) – The Vendor ID (VID) of the device in string format.
- **pid_s** (*str*) – The Product ID (PID) of the device in string format.

Returns *True* if the device is a Tempico, *False* otherwise.

Return type bool

verifyPyTempico (tuple_vid_pid)

Verifies whether the connected device is a Tempico device.

This function checks if the device's Vendor ID (VID) and Product ID (PID) match the values corresponding to a Tempico device. It returns *True* if the device is identified as a Tempico, and *False* otherwise.

Parameters **tuple_vid_pid** (*tuple*) – A tuple containing the VID and PID of the device.

Returns *True* if the device is a Tempico, *False* otherwise.

Return type bool

1.2 TempicoDevice class

class pyTempico.core.TempicoDevice (com_port)

Tausand Tempico TDC device object.

To create an object of the `TempicoDevice()` class, it is required to send as parameter the desired com_port. For example,

```
>>> my_tempico_device_object = pyTempico.TempicoDevice('COM5')
```

To modify or access attributes, **please use methods**. For example,

```
>>> my_tempico_device_object.getIdn()
```

To access attributes of a particular channel, use methods of the `TempicoChannel()` class through attributes ch1, ch2, ch3, ch4 of this class. For example, to get average cycles on channel 2,

```
>>> my_tempico_device_object.ch2.getAverageCycles()
```

or, as an alternative, send the channel number as a parameter

```
>>> my_tempico_device_object.getAverageCycles(2)
```

Changing attributes without using methods, do not change the actual parameters in the device.

Accessing attributes without using methods, returns values registered in local memory, that may not be updated.

To begin a measurement and read its results, use methods `measure()` and `fetch()`.

id_tempico_device

Unique identifier of the *TempicoDevice()* object.

Type int

ch1

Object of the *TempicoChannel()* class linked to TDC in channel 1 (input A).

Type *TempicoChannel*

ch2

Object of the *TempicoChannel()* class linked to TDC in channel 2 (input B).

Type *TempicoChannel*

ch3

Object of the *TempicoChannel()* class linked to TDC in channel 3 (input C).

Type *TempicoChannel*

ch4

Object of the *TempicoChannel()* class linked to TDC in channel 4 (input D).

Type *TempicoChannel*

device

Serial port object.

Type Serial

idn

Identification string.

Type str

number_of_channels

number of stop inputs of the device.

Type int

number_of_runs

Number of measurement runs of the TDCs in *TempicoDevice()*.

Type int

port

Serial port string.

Type str

threshold

Threshold voltage on the rising edge of start and stops inputs of TDCs in the *TempicoDevice()*.

Type float

abort (*validate=True*)

Cancels an ongoing measurement on the *TempicoDevice()*.

This function sends a cancel command to the *TempicoDevice()* to stop any measurement currently in progress. It ensures that all measurement processes are halted and the device is ready for a new operation or safely turned off.

This function requires that a connection is established with the *TempicoDevice()*.

Parameters **validate** (*bool*, *optional*) – If True, waits and validates if every channel applies the abort successfully. Default is True.

calibrateDelay()

Calibrates the internal delay.

This command adjusts the hardware's internal timing delay to ensure accurate measurements. Only supported on devices with hardware version "TP12".

Parameters **None** –

Returns **None**

cleanNumberList(number_list)

Removes non-valid elements of a Tempico dataset, given in number_list, a number 2D-list (read data). Returns a clean number 2D-list.

Validations applied:

- length of row
- type of data (integers, except for start_s register)
- ch in range
- run in range
- sorted stops
- stops in range
- sequential run values
- increasing start

The dataset of a *TempicoDevice()* is in the following format:

[[ch,run,start_s,stop_ps1,...,stop_psN],...,[ch,run,start_time_us,stop_ps1,...,stop_psN]]

where

- 'ch' indicates the TDC channel,
- 'run' goes from 1 to NumberOfRuns,
- 'start_s' is the epoch timestamp of start pulse, in seconds, with microseconds precision; this value overflows (go back to zero) after 2^{32} seconds,
- 'stop_ps1' is the measured precision timelapse between start and the 1st stop pulse, in picoseconds,
- 'N' is the NumberOfStops.

Parameters **number_list** (*list(number)*) – a complete dataset message converted.

Returns a cleaned complete dataset message converted.

Return type list(number)

close()

Ends (closes) a connection with a *TempicoDevice()*.

It is recommended to close connection at the end of a routine, to free the device's port for future use.

Parameters **(none)** –

closeTempico()

Ends (closes) a connection with a *TempicoDevice()*.

Same as method *close()*.

Parameters **(none)** –

convertReadDataToFloatList (*data_string*)

Converts a string with a read dataset message issued by a *TempicoDevice()*, into an float 2D-list.

The dataset of a *TempicoDevice()* is in the following format:

```
[[ch,run,start_s,stop_ps1,...,stop_psN],...,[ch,run,start_time_us,stop_ps1,...,stop_psN]]
```

where

- 'ch' indicates the TDC channel,
- 'run' goes from 1 to NumberOfRuns,
- 'start_s' is the epoch timestamp of start pulse, in seconds, with microseconds precision; this value overflows (go back to zero) after 2^{32} seconds,
- 'stop_ps1' is the measured precision timelapse between start and the 1st stop pulse, in picoseconds,
- 'N' is the NumberOfStops.

Every value in the dataset is converted to a float.

Parameters *data_string* (*str*) – dataset message to convert.

Returns dataset message converted.

Return type list(float)

convertReadDataToNumberList (*data_string*)

Converts a string with a read dataset message issued by a *TempicoDevice()*, into a number 2D-list (integer or float).

The dataset of a *TempicoDevice()* is in the following format:

```
[ [ch,run,start_s,stop_ps1,...,stop_psN], ..., [ch,run,start_time_us,stop_ps1,...,stop_psN] ]
```

where

- 'ch' (int) indicates the TDC channel,
- 'run' (int) goes from 1 to NumberOfRuns,
- 'start_s' (float) is the epoch timestamp of start pulse, in seconds, with microseconds precision; this value overflows (go back to zero) after 2^{32} seconds,
- 'stop_ps1' (int) is the measured precision timelapse between start and the 1st stop pulse, in picoseconds,
- 'N' (int) is the NumberOfStops.

Every value in the dataset is either an integer or a float.

Parameters *data_string* (*str*) – dataset message to convert.

Returns dataset message converted.

Return type list(number)

decrementGeneratorFrequency ()

Decreases the internal generator frequency by one step (only for TP12 devices).

This function reduces the generator frequency on a predefined decrement scale, $\{1, 2, 5\} \cdot 10^n$, depending on the current frequency.

Parameters **None** –

Returns None

decrementThresholdVoltage()

Reduces the threshold voltage on the rising edge of start and stops inputs of TDCs in the *TempicoDevice()*.

To validate the actual threshold voltage applied, method *getThresholdVoltage()* should be called.

This function requires that a connection is established with the *TempicoDevice()*.

Parameters (none) –

disableChannel(channel)

Disables the specified *TempicoChannel()*.

By default, channels are enabled.

This function requires that a connection is established with the *TempicoDevice()*, and delegates the disabling operation to the selected *TempicoChannel()*. If the channel is invalid, the function does nothing.

To validate the status of the channel, method *isEnabled()* may be used.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'-'D') to access and configure the corresponding *TempicoChannel()*.

Parameters channel (*int* or *str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

enableChannel(channel)

Enables the specified *TempicoChannel()*.

By default, channels are enabled.

This function requires that a connection is established with the *TempicoDevice()*, and delegates the enabling operation to the selected *TempicoChannel()*. If the channel is invalid, the function does nothing.

To validate the status of the channel, method *isEnabled()* may be used.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'-'D') to access and configure the corresponding *TempicoChannel()*.

Parameters channel (*int* or *str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

fetch(validate=True)

Reads the most recent measurement data set form a *TempicoDevice()*.

The dataset of a *TempicoDevice()* is in the following format:

```
[ [ch, run, start_s, stop_ps1, ..., stop_psN], ..., [ch, run, start_time_us, stop_ps1, ...  
↪, stop_psN] ]
```

where

- 'ch' (int) indicates the TDC channel,

- 'run' (int) goes from 1 to NumberOfRuns,
- 'start_s' (float) is the epoch timestamp of start pulse, in seconds with microsecond resolution. This value overflows (go back to zero) after 2^{32} seconds,
- 'stop_ps1' (int) is the measured precision timelapse between start and the 1st stop pulse, in picoseconds,
- 'N' (int) is the NumberOfStops.

Every value in the dataset is either an integer or a float number.

If no measurement has been done, the device may respond with an empty dataset. To make a new measurement, method `measure()` must be used.

This function requires that a connection is established with the `TempicoDevice()`.

Parameters `validate` (*bool, optional*) – If True, the measured dataset is validated, and invalid registers get cleaned. Default is True.

Returns measured dataset.

Return type list(number)

getAverageCycles (*channel*)

Returns the average cycles of the specified `TempicoChannel()`.

By default, average cycles = 1 (no multi-cycle averaging).

If the connection is established with the `TempicoDevice()`, this function delegates the query to the selected channel object and retrieves its average cycles value. If the connection is not open, or the channel is invalid, the function returns -1.

This version belongs to the `TempicoDevice()` class and requires specifying the channel number or label ('A'-'D') to access the corresponding `TempicoChannel()`.

Parameters `channel` (*int or str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

Returns Number of average cycles, or -1 if the channel is invalid.

Return type integer

getBaudRate ()

Returns the `TempicoDevice()` baud rate.

Parameters (*none*) –

Returns baud rate.

Return type int

getDateTime (*convert_to_datetime=False*)

Same as `getDatetime`

getDatetime (*convert_to_datetime=False*)

Returns the number of seconds since the Tempico device was powered on, based on its internal clock. If the device has been synchronized, the seconds count corresponds to the system time of the host PC.

This function sends the `DTIME?` command to the device, reads the response, and parses it into a float representing the elapsed time in seconds. If the device does not respond correctly, the function returns -1.

The timestamp resolution is in microseconds (μ s), and the count starts from power-up by default. Synchronization with the PC time must be previously configured on the device, otherwise the value is relative to the device's uptime.

Parameters `convert_to_datetime` (*bool*, *optional*) – If True, the value is returned as a datetime object. Default is False.

Returns Elapsed time in seconds since the device was powered on or synchronized with the PC clock. Returns -1 if no valid response is received.

Return type float

getDelay (*channel*)

Retrieves the internal delay for the specified channel, in picoseconds.

In TP10 devices, this value is zero. In TP12 devices, this value is read from the device.

The function selects the given channel and queries its current delay setting. If the channel is invalid, it returns -1.

Parameters `channel` (*int*) – channel number to read the delay from.

Returns internal delay of the stop channel with respect to the start, in picoseconds. Returns -1 when fails.

Return type float

getFirmware ()

Returns the *TempicoDevice* () firmware version.

Parameters (*none*) –

Returns firmware version.

Return type str

getGeneratorFrequency ()

Returns the frequency of the internal pulse generator (only for TP12 devices).

This function queries the device with the 'CONF:GEN:FREQ?' command to obtain the current frequency of the internal pulse generator. If the connection is not open, an informational message is printed. This feature is not available on TP1004 devices.

Parameters *None* –

Returns Current frequency of the internal pulse generator. Returns an empty string if it cannot be retrieved.

Return type float

getIdn ()

Returns the *TempicoDevice* () identification string.

If the connection is established with the *TempicoDevice* (), this function request the device for the string. If not, the most recent read string is returned.

Parameters (*none*) –

Returns identification string.

Return type str

getLastDelaySync (*convert_to_datetime=False*)

Returns the timestamp of the last delay calibration performed on the *TempicoDevice* ().

If the device is connected and the hardware version is “TP12”, the function requests the last delay calibration time using the ‘DTIME:LDElay?’ command. If *convert_to_datetime* is True, the value is returned as a datetime object.

Parameters *convert_to_datetime* (*bool*, *optional*) – If True, returns the timestamp as a datetime object. Default is False.

Returns Timestamp of the last delay calibration, as a Unix timestamp or datetime object. Returns -1 if it cannot be retrieved.

Return type float or datetime

getLastStart (*convert_to_datetime=False*)

Returns the datetime of the last start event registered by the *TempicoDevice()*.

If the connection is established with the *TempicoDevice()*, this function requests the device for the last start timestamp using the ‘DTIME:LStart?’ command. If not, the value -1 is returned.

The returned value corresponds to the timestamp of the most recent start event. If no start has occurred yet, the device returns 0. If *convert_to_datetime* is set to True, the value is returned as a datetime object instead of a float.

Parameters *convert_to_datetime* (*bool*, *optional*) – If True, the value is returned as a datetime object. Default is False.

Returns Timestamp of the last start event, either as a float (Unix timestamp) or as a datetime object if *convert_to_datetime* is True. Returns -1 if the value could not be retrieved.

Return type float or datetime

getLastSync (*convert_to_datetime=False*)

Returns the datetime of the last synchronization performed on the *TempicoDevice()*.

If the connection is established with the *TempicoDevice()*, this function requests the device for the last synchronization timestamp using the ‘DTIME:LSYNc?’ command. If not, the value -1 is returned.

The returned value corresponds to the timestamp of the most recent synchronization with the host system. If no synchronization has occurred yet, the device returns 0. If *convert_to_datetime* is set to True, the value is returned as a datetime object instead of a float.

Parameters *convert_to_datetime* (*bool*, *optional*) – If True, the value is returned as a datetime object. Default is False.

Returns Timestamp of the last synchronization event, either as a float (Unix timestamp) or as a datetime object if *convert_to_datetime* is True. Returns -1 if the value could not be retrieved.

Return type float or datetime

getMaximumDateTime (*convert_to_datetime=False*)

Same as getMaximumDatetime

getMaximumDatetime (*convert_to_datetime=False*)

Returns the maximum datetime value allowed by the *TempicoDevice()*.

If the connection is established with the *TempicoDevice()*, this function requests the device for the maximum allowed date using the ‘DTIME:MAXimum?’ command. If not, the value -1 is returned.

The returned value corresponds to the latest timestamp that can be set on the device without causing an error. If *convert_to_datetime* is set to True, the value is returned as a datetime object instead of a float.

Parameters *convert_to_datetime* (*bool*, *optional*) – If True, the value is returned as a datetime object. Default is False.

Returns Maximum allowed datetime, either as a float (Unix timestamp) or as a datetime object if `convert_to_datetime` is True.

Return type float or datetime

getMinimumDateTime (*convert_to_datetime=False*)

Same as `getMinimumDatetime`

getMinimumDatetime (*convert_to_datetime=False*)

Returns the minimum datetime value allowed by the `TempicoDevice()`.

If the connection is established with the `TempicoDevice()`, this function requests the device for the minimum allowed date using the 'DTIME:MINimum?' command. If not, the value -1 is returned.

The returned value corresponds to the earliest timestamp that can be set on the device without causing an error. If `convert_to_datetime` is set to True, the value is returned as a datetime object instead of a float.

Parameters `convert_to_datetime` (*bool, optional*) – If True, the value is returned as a datetime object. Default is False.

Returns Minimum allowed datetime, either as a float (Unix timestamp) or as a datetime object if `convert_to_datetime` is True.

Return type float or datetime

getMode (*channel*)

Returns the measurement mode of the specified `TempicoChannel()`.

By default, mode = 1.

If the connection is established with the `TempicoDevice()`, this function delegates the query to the selected channel object and retrieves its current measurement mode. If the connection is not open, or the channel is invalid, the function returns -1.

This version belongs to the `TempicoDevice()` class and requires specifying the channel number or label ('A'-'D') to access the corresponding `TempicoChannel()`.

Parameters `channel` (*int or str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

Returns

Mode. Possible values are:

In TP1004:

- 1: Short measurement range. Start-stop times from 12ns to 500ns.
- 2: Large measurement range. Start-stop times from 125ns to 4ms.

In TP1204:

- 1: Short measurement range. Start-stop times from -200ns to 300ns.
- 2: Large measurement range. Start-stop times from -50ns to 4ms.

Returns -1 if the channel is invalid.

Return type integer

getModelIdn ()

Returns the identifier of the model associated with the device.

Returns The internal model identifier (*model_idn*).

Return type str

getNumberOfRuns ()

Returns the number of measurement runs of the TDCs in *TempicoDevice ()*.

By default, number of runs = 1 (single measurement).

If the connection is established with the *TempicoDevice ()*, this function request the device for the value. If not, the most recent value is returned.

Parameters (none) –

Returns Number of number of runs.

Return type integer

getNumberOfStops (channel)

Returns the expected number of stop pulses for the specified *TempicoChannel ()*.

By default, number of stops = 1 (single start → single stop).

The TDC must receive all the expected number of stops to register them as a valid measurement; otherwise, the measurements are discarded.

If the connection is established with the *TempicoDevice ()*, this function delegates the query to the selected channel object and retrieves its number of stops. If the connection is not open, or the channel is invalid, the function returns -1.

This version belongs to the *TempicoDevice ()* class and requires specifying the channel number or label ('A'-'D') to access the corresponding *TempicoChannel ()*.

Parameters channel (*int or str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

Returns Number of stops, or -1 if the channel is invalid.

Return type integer

getOverflowParameter ()

Retrieves the overflow parameter.

The returned value depends on the device hardware version. For TP12 devices, the function returns -1000000; otherwise, it returns -1.

Parameters None –

Returns Overflow parameter value based on the hardware version.

Return type int

getSerialNumber ()

Returns the serial number of the connected *TempicoDevice ()*.

This function searches for the serial number associated with the current COM port used by the device. If the connection is established, it queries the list of available serial ports to match the active one and extract its serial number.

If the connection is not open, a message is printed and an empty string is returned.

Parameters (**none**) –

Returns Serial number of the device, or empty string if not found or if the connection is not open.
e.g: “TP1004-220500”

Return type string

getSettings ()

Reads the current settings form a *TempicoDevice* ().

The response for settings query on a *TempicoDevice* () is in the following format:

```
CH1:ACYC 1;CH1:ENAB 1;CH1:NST 1;...;CH4:STOP:MASK 0.0000;NRUN 1;THR 1.00
```

This function requires that a connection is established with the *TempicoDevice* ().

Parameters (**none**) –

Returns device settings.

Return type str

getStartEdge (channel)

Returns the edge type used on start pulses of the specified *TempicoChannel* ().

By default, start edge = ‘RISE’.

If the connection is established with the *TempicoDevice* (), this function delegates the query to the selected channel object and retrieves its configured start edge type. If the connection is not open, or the channel is invalid, the function returns -1.

This version belongs to the *TempicoDevice* () class and requires specifying the channel number or label (‘A’–‘D’) to access the corresponding *TempicoChannel* ().

Parameters **channel** (*int or str*) – Channel identifier. Accepted values are:

- 1 or ‘A’ or ‘a’
- 2 or ‘B’ or ‘b’
- 3 or ‘C’ or ‘c’
- 4 or ‘D’ or ‘d’

Returns

Start edge type. Possible values are:

- ‘RISE’: TDC timing starts on a rising edge of the start pulse.
- ‘FALL’: TDC timing starts on a falling edge of the start pulse.

Returns -1 if the channel is invalid.

Return type string or int

getStartSource (channel)

Returns the start signal source for the specified channel.

This function checks whether the start signal of the selected channel comes from the internal generator or from an external input connected to the device.

Parameters `channel` (*int*) – Channel number to read the start source from.

Returns Start source identifier. Returns -1 if the channel is invalid.

Return type `int`

getState (*channel*)

Returns the state of the specified *TempicoChannel()*.

This function is used to validate if a reset or an abort command has been successfully applied.

Some possible states are:

- 0: disabled.
- 1: idle, enabled.
- 10: processing a reset.
- 11: processing an abort.

other states are related with the measurement process.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'-'D') to access the corresponding *TempicoChannel()*.

Parameters `channel` (*int or str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

Returns

state.

Returns -1 if the channel is invalid.

Return type `int`

getStates ()

Returns a list with the state of each *TempicoChannel()* within *TempicoDevice()*.

This function is used to validate if a reset or an abort command has been successfully applied.

Some possible states are:

- 0: disabled.
- 1: idle, enabled.
- 10: processing a reset.
- 11: processing an abort.

other states are related with the measurement process.

Parameters (*none*) –

Returns states per channel.

Return type `list(integer)`

getStatus (*channel*)

Returns the internal status of the specified *TempicoChannel()*.

This function is used to obtain the state of a channel, used to validate if a reset or an abort command has been successfully applied.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'-'D') to access the corresponding *TempicoChannel()*.

Parameters **channel** (*int or str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

Returns

status fields and values.

Returns -1 if the channel is invalid.

Return type dict

getStopEdge (*channel*)

Returns the edge type used on stop pulses of the specified *TempicoChannel()*.

By default, stop edge = 'RISE'.

If the connection is established with the *TempicoDevice()*, this function delegates the query to the selected channel object and retrieves its configured stop edge type. If the connection is not open, or the channel is invalid, the function returns -1.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'-'D') to access the corresponding *TempicoChannel()*.

Parameters **channel** (*int or str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

Returns

Stop edge type. Possible values are:

- 'RISE': TDC timing ends on a rising edge of the stop pulse.
- 'FALL': TDC timing ends on a falling edge of the stop pulse.

Returns -1 if the channel is invalid.

Return type string or int

getStopMask (*channel*)

Returns the stop mask time (in microseconds) of the specified *TempicoChannel()*.

By default, in TP1004 stop mask = 0us (no masking). By default, in TP1204 stop mask = -0.25us (no masking).

The stop mask defines the period after receiving a start pulse during which stop pulses are ignored. This helps eliminate unwanted noise or early pulses.

If the connection is established with the `TempicoDevice()`, this function delegates the query to the selected channel object and retrieves its current stop mask time. If the connection is not open, or the channel is invalid, the function returns -1.

This version belongs to the `TempicoDevice()` class and requires specifying the channel number or label ('A'-'D') to access the corresponding `TempicoChannel()`.

Parameters `channel` (*int or str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

Returns Stop mask time in microseconds, or -1 if the channel is invalid.

Return type float

getStopMaskMinimum()

Retrieves the minimum stop mask value, in microseconds.

The returned value depends on the device hardware version. For TP12 devices, the function returns -0.25; otherwise, it returns 0.

Parameters `None` –

Returns minimum stop mask value based on the hardware version.

Return type float

getStopSource(channel)

Returns the stop signal source for the specified channel.

This function checks whether the stop signal of the selected channel comes from the internal generator or from an external input connected to the device.

Parameters `channel` (*int*) – Channel number to read the stop source from.

Returns Stop source identifier. Returns -1 if the channel is invalid.

Return type int

getTempicoChannel(channel)

Returns the `TempicoChannel()` object corresponding to the specified channel.

This function allows selecting a channel by its number (1–4) or label ('A'-'D'). If the input is valid, it returns the corresponding `TempicoChannel()` instance. If the input is invalid, an error message is printed and -1 is returned.

Parameters `channel` (*int or str*) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

Returns The corresponding `TempicoChannel()` object if the input is valid, or -1 if the channel is invalid.

Return type *TempicoChannel* or int

getThresholdVoltage()

Returns the threshold voltage on the rising edge of start and stops inputs of TDCs in the *TempicoDevice()*.

Start and stop inputs are coupled to 50 ohms.

By default, threshold voltage = 1.00V (recommended for TTL>2.5V).

All inputs are 5V tolerant.

Gate input. This parameter does not have effect on the gate input. Gate input accepts 3.3V TTL and 5V TTL signals.

- When gate is disconnected, system is enabled.
- When gate is connected to 0V, system is disabled.
- When gate is connected to 3.3V/5V, system is enabled.

If the connection is established with the *TempicoDevice()*, this function request the device for the value. If not, the most recent value is returned.

Parameters (none) –

Returns start and stop inputs threshold voltage.

Return type float

incrementGeneratorFrequency()

Increases the internal generator frequency by one step (only for TP12 devices).

This function raises the generator frequency on a predefined increment scale, $\{1, 2, 5\} \cdot 10^n$, depending on the current frequency.

Parameters None –

Returns None

incrementThresholdVoltage()

Increases the threshold voltage on the rising edge of start and stops inputs of TDCs in the *TempicoDevice()*.

To validate the actual threshold voltage applied, method *getThresholdVoltage()* should be called.

This function requires that a connection is established with the *TempicoDevice()*.

Parameters (none) –

isEnabled(channel)

Returns whether the specified *TempicoChannel()* is enabled.

By default, channels are enabled.

If the connection is established with the *TempicoDevice()*, this function delegates the query to the selected channel object and retrieves its current enable status. If the connection is not open, or the channel is invalid, the function returns -1.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'–'D') to access and query the corresponding *TempicoChannel()*.

Parameters **channel** (int or str) – Channel identifier. Accepted values are:

- 1 or 'A' or 'a'
- 2 or 'B' or 'b'

- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

Returns True if the channel is enabled, False if disabled, or -1 if the channel is invalid.

Return type bool or int

isIdle()

Finds if every channel in the *TempicoDevice()* is in the idle state.

This method is used to validate if a reset command has been successfully applied.

Returns True if every channel is idle.

Return type bool

isIdleOrDisabled()

Finds if every channel in the *TempicoDevice()* is in the idle state or in the disabled state.

This method is used to validate if an abort command has been successfully applied.

Returns True if every channel is either idle or disabled.

Return type bool

isOpen()

Returns if a TDC *TempicoDevice()* connection is established (open).

Parameters (none) –

Returns True when *TempicoDevice()* connection is open.

Return type bool

isPendingReadMessage()

Determines if a pending message is available to be read in a *TempicoDevice()* serial port.

Parameters (none) –

Returns True, when a pending message is found.

Return type bool

isValidConsecutiveNumberListRows(row, next_row)

Pair of consecutive rows validation of a number 2D-list (read data).

Validations applied:

- length of next_row
- sequential run values
- increasing start

A single row of the dataset of a *TempicoDevice()* is in the following format:

[ch,run,start_s,stop_ps1,...,stop_psN]

where

- 'ch' indicates the TDC channel,
- 'run' goes from 1 to NumberOfRuns,
- 'start_s' is the epoch timestamp of start pulse, in seconds, with microseconds precision; this value overflows (go back to zero) after 2^{32} seconds,
- 'stop_ps1' is the measured precision timelapse between start and the 1st stop pulse, in picoseconds,

- 'N' is the NumberOfStops.

Parameters

- **row** (*list (number)*) – single line of dataset message converted.
- **next_row** (*list (number)*) – next single line of dataset message converted.

Returns True when pair of rows passes all validations.

Return type bool

isValidNumberListRow (*row*)

Single row validation of a number 2D-list (read data).

Validations applied:

- length of row
- type of data (integers, except for start_s register)
- ch in range
- run in range
- sorted stops
- stops in range

A single row of the dataset of a *TempicoDevice()* is in the following format:

[ch,run,start_s,stop_ps1,...,stop_psN]

where

- 'ch' indicates the TDC channel,
- 'run' goes from 1 to NumberOfRuns,
- 'start_s' is the epoch timestamp of start pulse, in seconds, with microseconds precision; this value overflows (go back to zero) after 2³² seconds,
- 'stop_ps1' is the measured precision timelapse between start and the 1st stop pulse, in picoseconds,
- 'N' is the NumberOfStops.

Parameters **row** (*list (number)*) – single line of dataset message converted.

Returns True when single row passes all validations.

Return type bool

measure (*validate=True*)

Begins a measurement sequence and reads its dataset from a *TempicoDevice()*.

The dataset of a *TempicoDevice()* is in the following format:

```
[ [ch, run, start_s, stop_ps1, ..., stop_psN], ..., [ch, run, start_time_us, stop_ps1, ...  
↪, stop_psN] ]
```

where

- 'ch' (int) indicates the TDC channel,
- 'run' (int) goes from 1 to NumberOfRuns,

- 'start_s' (float) is the epoch timestamp of start pulse, in seconds with microsecond resolution. This value overflows (go back to zero) after 2^{32} seconds,
- 'stop_ps1' (int) is the measured precision timelapse between start and the 1st stop pulse, in picoseconds,
- 'N' (int) is the NumberOfStops.

Every value in the dataset is either an integer or a float number.

If measurement cannot be completed within timeout, the device may respond with an incomplete or empty dataset. In this case, to obtain a complete dataset, the method `fetch()` may be called later.

This function requires that a connection is established with the `TempicoDevice()`.

Parameters `validate` (*bool, optional*) – If True, the measured dataset is validated, and invalid registers get cleaned. Default is True.

Returns measured dataset.

Return type list(number)

open()

Establishes (opens) a connection with a `TempicoDevice()`.

It is mandatory to establish a connection with this method before to be able to send/to receive data to/from the device.

Parameters (*none*) –

openTempico()

Establishes (opens) a connection with a `TempicoDevice()`.

Same as method `open()`.

Parameters (*none*) –

readIdnFromDevice()

Returns the `TempicoDevice()` identification string, by requesting it to the device.

This function requires that a connection is established with the `TempicoDevice()`. As an alternative, method `getIdn()` may be used.

Parameters (*none*) –

Returns identification string.

Return type str

readMessage()

Reads pending messages sent by a `TempicoDevice()` from its serial port.

If no message is received, it waits the port timeout, typically 1s.

This function requires that a connection is established with the `TempicoDevice()`.

Parameters (*none*) –

Returns read message.

Return type str

reset (*validate=True*)

Sends a reset command to the `TempicoDevice()`.

Applying a reset clears all the settings of the `TempicoDevice()` and its TempicoChannels to their default values.

This function requires that a connection is established with the `TempicoDevice()`.

Parameters `validate (bool, optional)` – If True, waits and validates if every channel applies the reset successfully. Default is True.

selfTest ()

Performs a self-test on the `TempicoDevice ()` hardware.

This function initiates a self-diagnostic test on the `TempicoDevice ()` to verify its hardware integrity. If the self-test is successful, it prints the message “Self test passed. Device is working properly.” If the self-test fails, it prints the message “Self test failed. Device may have a problem.”, indicating a potential issue with the hardware that may require further investigation or support.

This function requires that a connection is established with the `TempicoDevice ()`.

Parameters (`none`) –

Returns None

setAverageCycles (channel, averageCycles)

Modifies the average cycles of the specified `TempicoChannel ()`.

By default, average cycles = 1 (no multi-cycle averaging).

This function requires that a connection is established with the `TempicoDevice ()`, and delegates the configuration to the selected `TempicoChannel ()`. If the channel is invalid, the function does nothing.

This version belongs to the `TempicoDevice ()` class and requires specifying the channel number or label ('A'-'D') to access and configure the corresponding `TempicoChannel ()`.

Parameters

- **channel** (`int` or `str`) – Channel identifier. Accepted values are:
 - 1 or 'A' or 'a'
 - 2 or 'B' or 'b'
 - 3 or 'C' or 'c'
 - 4 or 'D' or 'd'
- **averageCycles** (`int`) – Desired average cycles for the TDC. Valid values are: 1, 2, 4, 8, 16, 32, 64, 128.

setDateTime (timeStampDateTime=None)

Same as `setDatetime`

setDatetime (timeStampDateTime=None)

Sets the internal clock of the Tempico device to a specified timestamp or to the current system time.

Sends a `DTIME <timestamp>` command using either the provided `timeStampDateTime` or the current time in Unix format with microsecond precision. If no response is received from the device, the function verifies the update by comparing timestamps.

Parameters `timeStampDateTime (float, optional)` – Unix timestamp (in seconds) to set on the device. If not provided, the current system time is used.

Returns None

setGeneratorFrequency (desired_frequency)

Sets the frequency of the internal pulse generator (only for TP12 devices).

This function changes the frequency of the internal generator using the 'CONF:GEN:FREQ' command. The value must be between 10 Hz and 10 000 000 Hz. The command is verified by reading back the applied frequency and checking that the relative error is below 0.5%. If the connection is not open or the hardware version is not supported, an informational message is printed.

Parameters `desired_frequency` (*float*) – Desired generator frequency in hertz (10 Hz – 10 000 000 Hz).

Returns None

setMaximumDateTime (*maximum_datetime*)

Same as setMaximumDatetime

setMaximumDatetime (*maximum_datetime*)

Sets the maximum datetime value allowed by the *TempicoDevice()*.

If the connection is established with the *TempicoDevice()*, this function sends the 'DTIME:MAXimum<timestamp>' command to update the maximum allowed timestamp. If the device accepts the value, it should return a confirmation response. If no response is received, the value is validated by re-reading it from the device.

Parameters `maximum_datetime` (*float*) – New maximum timestamp to configure on the device.

Returns None

setMinimumDateTime (*minimum_datetime*)

Same as setMinimumDatetime

setMinimumDatetime (*minimum_datetime*)

Sets the minimum datetime value allowed by the *TempicoDevice()*.

If the connection is established with the *TempicoDevice()*, this function sends the 'DTIME:MINimum<timestamp>' command to update the minimum allowed timestamp. If the device accepts the value, it should return a confirmation response. If no response is received, the value is validated by re-reading it from the device.

Parameters `minimum_datetime` (*float*) – New minimum timestamp to configure on the device.

Returns None

setMode (*channel, mode*)

Modifies the measurement mode of the specified *TempicoChannel()*.

By default, mode = 1. Possible values are:

In TP1004:

- 1: Short measurement range. Start-stop times from 12ns to 500ns.
- 2: Large measurement range. Start-stop times from 125ns to 4ms.

In TP1204:

- 1: Short measurement range. Start-stop times from -200ns to 300ns.
- 2: Large measurement range. Start-stop times from -50ns to 4ms.

This function requires that a connection is established with the *TempicoDevice()*, and delegates the configuration to the selected *TempicoChannel()*. If the channel is invalid, the function does nothing.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'-'D') to access and configure the corresponding *TempicoChannel()*.

Parameters

- **channel** (*int* or *str*) – Channel identifier. Accepted values are:
 - 1 or 'A' or 'a'

- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'

- **mode** (*int*) – Desired measurement mode for the TDC. Valid values are 1 or 2.

setNumberOfRuns (*number*)

Modifies the number of measurement runs of the TDCs in *TempicoDevice()*.

By default, number of runs = 1 (single measurement).

This function requires that a connection is established with the *TempicoDevice()*.

Parameters **number** (*int*) – desired number of runs for every TDC. Valid values are from 1 to 1000.

setNumberOfStops (*channel, numberOfStops*)

Modifies the expected number of stop pulses of the specified *TempicoChannel()*.

By default, number of stops = 1 (single start → single stop).

The TDC must receive all the expected number of stops to register them as a valid measurement; otherwise, the measurements are discarded. For extending the valid time range, consider using measurement mode 2.

This function requires that a connection is established with the *TempicoDevice()*, and delegates the configuration to the selected *TempicoChannel()*. If the channel is invalid, the function does nothing.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'-'D') to access and configure the corresponding *TempicoChannel()*.

Parameters

- **channel** (*int or str*) – Channel identifier. Accepted values are:
 - 1 or 'A' or 'a'
 - 2 or 'B' or 'b'
 - 3 or 'C' or 'c'
 - 4 or 'D' or 'd'
- **numberOfStops** (*int*) – Desired number of stops for the TDC. Valid values are from 1 to 5.

setStartEdge (*channel, edge_type*)

Sets the edge type used on start pulses of the specified *TempicoChannel()*.

By default, start edge = 'RISE'. Possible values are:

- 'RISE': TDC timing starts on a rising edge of the start pulse.
- 'FALL': TDC timing starts on a falling edge of the start pulse.

This function requires that a connection is established with the *TempicoDevice()*, and delegates the configuration to the selected *TempicoChannel()*. If the channel is invalid, the function does nothing.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'-'D') to access and configure the corresponding *TempicoChannel()*.

Parameters

- **channel** (*int or str*) – Channel identifier. Accepted values are:
 - 1 or 'A' or 'a'

- 2 or 'B' or 'b'
- 3 or 'C' or 'c'
- 4 or 'D' or 'd'
- **edge_type** (*str or int*) – Desired start edge type for the TDC. Accepted values are 'RISE', 1, 'RIS' or 'FALL', 0, 'FAL'.

setStartExternalSource (*channel*)

Sets the start signal source to external for the specified channel.

This function configures the selected channel so that its start signal comes from an external input instead of the internal generator.

Parameters **channel** (*int*) – Channel number to configure.

Returns None

setStartInternalSource (*channel*)

Sets the start signal source to the internal generator for the specified channel.

This function configures the selected channel so that its start signal is provided by the internal pulse generator instead of an external input.

Parameters **channel** (*int*) – Channel number to configure.

Returns None

setStopEdge (*channel, edge_type*)

Sets the edge type used on stop pulses of the specified *TempicoChannel()*.

By default, stop edge = 'RISE'. Possible values are:

- 'RISE': TDC timing ends on a rising edge of the stop pulse.
- 'FALL': TDC timing ends on a falling edge of the stop pulse.

This function requires that a connection is established with the *TempicoDevice()*, and delegates the configuration to the selected *TempicoChannel()*. If the channel is invalid, the function does nothing.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'–'D') to access and configure the corresponding *TempicoChannel()*.

Parameters

- **channel** (*int or str*) – Channel identifier. Accepted values are:
 - 1 or 'A' or 'a'
 - 2 or 'B' or 'b'
 - 3 or 'C' or 'c'
 - 4 or 'D' or 'd'
- **edge_type** (*str or int*) – Desired stop edge type for the TDC. Accepted values are 'RISE', 1, 'RIS' or 'FALL', 0, 'FAL'.

setStopExternalSource (*channel*)

Sets the stop signal source to external for the specified channel.

This function configures the selected channel so that its stop signal comes from an external input instead of the internal generator.

Parameters **channel** (*int*) – Channel number to configure.

Returns None

setStopInternalSource (*channel*)

Sets the stop signal source to the internal generator for the specified channel.

This function configures the selected channel so that its stop signal is provided by the internal pulse generator instead of an external input.

Parameters **channel** (*int*) – Channel number to configure.

Returns None

setStopMask (*channel, stop_mask_in_us*)

Modifies the stop mask time of the specified *TempicoChannel()*.

By default, in TP1004 stop mask = 0us (no masking). By default, in TP1204 stop mask = -0.25us (no masking).

The stop mask defines the period (in microseconds) after receiving a start pulse during which stop pulses are ignored. This can help suppress unwanted noise or early signals.

This function requires that a connection is established with the *TempicoDevice()*, and delegates the configuration to the selected *TempicoChannel()*. If the channel is invalid, the function does nothing.

This version belongs to the *TempicoDevice()* class and requires specifying the channel number or label ('A'-'D') to access and configure the corresponding *TempicoChannel()*.

Parameters

- **channel** (*int or str*) – Channel identifier. Accepted values are:
 - 1 or 'A' or 'a'
 - 2 or 'B' or 'b'
 - 3 or 'C' or 'c'
 - 4 or 'D' or 'd'
- **stop_mask_in_us** (*float*) – Desired stop mask time in microseconds. Valid values are from -0.25 to 4000.

setThresholdVoltage (*desired_voltage*)

Changes the threshold voltage on the rising edge of start and stops inputs of TDCs in the *TempicoDevice()*.

Start and stop inputs are coupled to 50 ohms.

By default, threshold voltage = 1.00V (recommended for TTL>2.5V).

All inputs are 5V tolerant.

Gate input. This parameter does not have effect on the gate input. Gate input accepts 3.3V TTL and 5V TTL signals.

- When gate is disconnected, system is enabled.
- When gate is connected to 0V, system is disabled.
- When gate is connected to 3.3V/5V, system is enabled.

To validate the actual threshold voltage applied, method *getThresholdVoltage()* should be called.

This function requires that a connection is established with the *TempicoDevice()*.

Parameters **desired_voltage** (*float*) – desired start and stop inputs threshold voltage.
Valid parameters are MINimum/MAXimum/DOWN/UP or a number from 0.90 to 1.60.

setThresholdVoltageToMaximum()

Sets to the maximum valid value the threshold voltage on the rising edge of start and stops inputs of TDCs in the *TempicoDevice()*.

To validate the actual threshold voltage applied, method *getThresholdVoltage()* should be called.

This function requires that a connection is established with the *TempicoDevice()*.

Parameters (none) –

setThresholdVoltageToMinimum()

Sets to the minimum valid value the threshold voltage on the rising edge of start and stops inputs of TDCs in the *TempicoDevice()*.

To validate the actual threshold voltage applied, method *getThresholdVoltage()* should be called.

This function requires that a connection is established with the *TempicoDevice()*.

Parameters (none) –

waitAndReadMessage(wait_time_ms=1)

Waits the specified time, and then reads pending messages sent by a *TempicoDevice()* from its serial port, if any.

If no message is received, it does not wait for a port timeout.

This function requires that a connection is established with the *TempicoDevice()*.

Parameters **wait_time_ms** (*int*, *optional*) – Waiting time, in milliseconds. Defaults to 1.

Returns read message.

Return type str

writeMessage(message)

Writes a message to a *TempicoDevice()* in its serial port.

If a response is expected after writing a message, the *readMessage()* method should be called afterwards to obtain the response.

This function requires that a connection is established with the *TempicoDevice()*.

Parameters **message** (*str*) – message to be sent.

1.3 TempicoChannel class

class pyTempico.core.TempicoChannel(*id_device*, *ch_num*)

Single channel on a Tempico Device.

To modify or access attributes, **please use methods**. For example, to get average cycles on channel 2,

```
>>> my_tempico_device_object.ch2.getAverageCycles()
```

or, as an alternative, send the channel number as a parameter

```
>>> my_tempico_device_object.getAverageCycles(2)
```

Changing attributes without using methods, do not change the actual parameters in the device.

Accessing attributes without using methods, returns values registered in local memory that may not be updated.

id_tempico_channel

Unique identifier for a *TempicoChannel()* object.

Type int

id_tempico_device

Identifier of the *TempicoDevice()* linked to this *TempicoChannel()* object.

Type int

average_cycles

Average cycles.

Type int

channel_number

Number of the channel in the device (1=A, 2=B,...).

Type int

enable

True when the channel is enabled.

Type bool

mode

Measurement mode. 1|2.

- 1: Short measurement range. Start-stop times from 12ns to 500ns.
- 2: Large measurement range. Start-stop times from 125ns to 4ms.

Type int

number_of_stops

Number of stop pulses expected after a start pulse arrives. 1..5.

Type int

parent_tempico_device

Pointer reference to parent object of *TempicoDevice()* class.

Type *TempicoDevice*

start_edge

Edge type on the start pulse used to begin timing. RISE|FALL.

Type str

stop_edge

Edge type on the stop pulses used to end timing. RISE|FALL.

Type str

stop_mask

Time that stop pulses are ignored after receiving a start pulse on the TDC. Value in microseconds. 0..4000.

Type int

disableChannel()

Disables a TDC *TempicoChannel()*.

By default, channels are enabled.

This function requires that a connection is established with the *TempicoDevice()* of the *TempicoChannel()*.

To validate the status of the channel, method `isEnabled()` may be used.

Parameters (none) –

enableChannel()

Enables a TDC `TempicoChannel()`.

By default, channels are enabled.

This function requires that a connection is established with the `TempicoDevice()` of the `TempicoChannel()`.

To validate the status of the channel, method `isEnabled()` may be used.

Parameters (none) –

getAverageCycles()

Returns the average cycles of the TDC `TempicoChannel()`.

By default, average cycles = 1 (no multi-cycle averaging).

If the connection is established with the `TempicoDevice()`, this function request the device for the value. If not, the most recent value is returned.

Parameters (none) –

Returns Number of average cycles.

Return type integer

getDelay()

Returns the internal delay value for this channel, in picoseconds.

In TP10 devices, this value is zero. In TP12 devices, this value is read from the device.

This function queries the device using the 'CONFIGure:CHx:DElay?' command to obtain the internal delay assigned to the current channel. If the connection is not open or the hardware version is not supported, an informational message is printed.

Parameters None –

Returns internal delay of the stop channel with respect to the start, in picoseconds. Returns -1 when fails.

Return type float

getMode()

Returns the measurement mode of the TDC `TempicoChannel()`.

By default, mode = 1.

If the connection is established with the `TempicoDevice()`, this function request the device for the value. If not, the most recent value is returned.

Parameters (none) –

Returns

Mode. Possible values are,

In TP1004:

- 1: Short measurement range. Start-stop times from 12ns to 500ns.
- 2: Large measurement range. Start-stop times from 125ns to 4ms.

In TP1204:

- 1: Short measurement range. Start-stop times from -200ns to 300ns.
- 2: Large measurement range. Start-stop times from -50ns to 4ms.

Return type integer

getNumberOfStops ()

Returns the expected number of stop pulses of the TDC *TempicoChannel ()*.

By default, number of stops = 1 (single start -> single stop).

The TDC must receive all the expected number of stops to register them as a valid measurement; otherwise, the measurements are discarded.

If the connection is established with the *TempicoDevice ()*, this function request the device for the value. If not, the most recent value is returned.

Parameters (none) –

Returns Number of stops.

Return type integer

getStartEdge ()

Returns the edge type used on start pulses of the TDC *TempicoChannel ()*.

By default, start edge = 'RISE'.

If the connection is established with the *TempicoDevice ()*, this function request the device for the value. If not, the most recent value is returned.

Parameters (none) –

Returns

start edge type. Possible values are,

- 'RISE': TDC timing starts on a rising edge of the start pulse.
- 'FALL': TDC timing starts on a falling edge of the start pulse.

Return type string

getStartSource ()

Returns the start signal source for this channel.

This function determines whether the start signal of the current channel comes from the internal generator or from an external input.

Internal pulse generator is available in TP12 devices.

Parameters None –

Returns Signal source identifier ("INternal" or "EXternal").

Return type str

getState ()

Returns the state of a *TempicoChannel ()*.

This function is used to validate if a reset or an abort command has been successfully applied.

Some possible states are:

- 0: disabled.
- 1: idle, enabled.
- 10: processing a reset.

- 11: processing an abort.

other states are related with the measurement process.

Parameters (**none**) –

Returns state.

Return type int

getStatus()

Returns the internal status of a *TempicoChannel()*.

This function is used to obtain the state of a channel, used to validate if a reset or an abort command has been successfully applied.

Parameters (**none**) –

Returns status fields and values.

Return type dict

getStopEdge()

Returns the edge type used on stop pulses of the TDC *TempicoChannel()*.

By default, stop edge = 'RISE'.

If the connection is established with the *TempicoDevice()*, this function request the device for the value. If not, the most recent value is returned.

Parameters (**none**) –

Returns

stop edge type. Possible values are,

- 'RISE': TDC timing ends on a rising edge of the stop pulse.
- 'FALL': TDC timing ends on a falling edge of the stop pulse.

Return type string

getStopMask()

Returns the time that stop pulses are ignored after receiving a start pulse on the TDC *TempicoChannel()*. In microseconds.

By default, in TP1004 stop mask = 0us (no masking). By default, in TP1204 stop mask = -0.25us (no masking).

If the connection is established with the *TempicoDevice()*, this function request the device for the value. If not, the most recent value is returned.

Parameters (**none**) –

Returns stop mask time, in microseconds.

Return type float

getStopSource()

Returns the stop signal source for this channel.

This function determines whether the stop signal of the current channel comes from the internal generator or from an external input.

Internal pulse generator is available in TP12 devices.

Parameters **None** –

Returns Signal source identifier (“INternal” or “EXternal”).

Return type str

isEnabled()

Returns if a TDC *TempicoChannel()* is enabled.

By default, channels are enabled.

If the connection is established with the *TempicoDevice()*, this function request the device for the value. If not, the most recent value is returned.

Parameters (none) –

Returns True, when TDC channel is enabled.

Return type bool

setAverageCycles(number)

Modifies the average cycles of the TDC *TempicoChannel()*.

By default, average cycles = 1 (no multi-cycle averaging).

This function requires that a connection is established with the *TempicoDevice()* of the *TempicoChannel()*.

Parameters **number** (int) – desired average cycles for the TDC. Valid values are 1|2|4|8|16|32|64|128.

setMode(number)

Modifies the measurement mode of the TDC *TempicoChannel()*.

By default, mode = 1. Possible values are,

In TP1004:

- 1: Short measurement range. Start-stop times from 12ns to 500ns.
- 2: Large measurement range. Start-stop times from 125ns to 4ms.

In TP1204:

- 1: Short measurement range. Start-stop times from -200ns to 300ns.
- 2: Large measurement range. Start-stop times from -50ns to 4ms.

This function requires that a connection is established with the *TempicoDevice()* of the *TempicoChannel()*.

Parameters **number** (int) – desired measurement mode for the TDC. Valid values are 1 or 2.

setNumberOfStops(number)

Modifies the expected number of stop pulses of the TDC *TempicoChannel()*.

By default, number of stops = 1 (single start -> single stop).

The TDC must receive all the expected number of stops to register them as a valid measurement; otherwise, the measurements are discarded. For extending the valid time range, consider using measurement mode 2.

This function requires that a connection is established with the *TempicoDevice()* of the *TempicoChannel()*.

Parameters **number** (int) – desired number of stops for the TDC. Valid values are from 1 to 5.

setStartEdge(edge_type)

Sets the edge type used on start pulses of the TDC *TempicoChannel()*.

By default, start edge = ‘RISE’. Possible values are,

- 'RISE': TDC timing starts on a rising edge of the start pulse.
- 'FALL': TDC timing starts on a falling edge of the start pulse.

This function requires that a connection is established with the `TempicoDevice()` of the `TempicoChannel()`.

Parameters `edge_type` (*str*) – desired start edge type for the TDC. Valid values are 'RISE', 1, 'FALL', 0.

setStartExternalSource()

Sets the start signal source to external (only for TP12 devices).

This function configures the start input of the current channel so that its signal comes from an external source instead of the internal generator.

Parameters `None` –

Returns `None`

setStartInternalSource()

Sets the start signal source to the internal generator (only for TP12 devices).

This function configures the start input of the current channel so that its signal is provided by the internal pulse generator instead of an external source.

Parameters `None` –

Returns `None`

setStopEdge (*edge_type*)

Sets the edge type used on stop pulses of the TDC `TempicoChannel()`.

By default, stop edge = 'RISE'. Possible values are,

- 'RISE': TDC timing ends on a rising edge of the stop pulse.
- 'FALL': TDC timing ends on a falling edge of the stop pulse.

This function requires that a connection is established with the `TempicoDevice()` of the `TempicoChannel()`.

Parameters `edge_type` (*str*) – desired stop edge type for the TDC. Valid values are 'RISE', 1, 'FALL', 0.

setStopExternalSource()

Sets the stop signal source to external (only for TP12 devices).

This function configures the stop input of the current channel so that its signal comes from an external source instead of the internal generator.

Parameters `None` –

Returns `None`

setStopInternalSource()

Sets the stop signal source to the internal generator (only for TP12 devices).

This function configures the stop input of the current channel so that its signal is provided by the internal pulse generator instead of an external source.

Parameters `None` –

Returns `None`

setStopMask (*stop_mask_in_us*)

Modifies the time that stop pulses are ignored after receiving a start pulse on the TDC *TempicoChannel()*.

By default, in TP1004 stop mask = 0us (no masking). By default, in TP1204 stop mask = -0.25us (no masking).

This function requires that a connection is established with the *TempicoDevice()* of the *TempicoChannel()*.

Parameters **stop_mask_in_us** (*float*) – desired stop mask for the TDC, in microseconds.
Valid values are from -0.25 to 4000.

EXAMPLES

2.1 Settings examples

Examples on how to read and change settings on a Tausand Tempico.

2.1.1 readIdentifiersExample.py

```
# -*- coding: utf-8 -*-
"""readIdentifiersExample

    Created on Thu Jan 29 11:03 2026

    Connects to a Tausand Tempico device and reads its identifier strings.

    Instructions:
        * before running this example, pyTempico package must be installed.
        * change 'my_port' to your corresponding port.
        * run.

    You do not need a connected signal to run this example.

    This example:
        * opens connection to the device,
        * uses get methods to read identifier strings,
        * closes connection.

    | @author: David Guzman, Tausand Electronics
    | dguzman@tausand.com
    | https://www.tausand.com
"""
import pyTempico

my_port = 'COM7' #change this port to your Tempico device's port

my_device = pyTempico.TempicoDevice(my_port)    #create object

#Open connection
print('opening connection with device in port',my_port)
my_device.open()    #open connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
```

(continues on next page)

(continued from previous page)

```

    print('connection with my_device is close')

#Use get methods to obtain identifier strings of a Tausand Tempico device
my_idn = my_device.getIdn()
my_model_idn = my_device.getModelIdn()
my_serial_number = my_device.getSerialNumber()
my_firmware = my_device.getFirmware()

#Print string identifiers
print('\nIDN string:\t\t',my_idn)
print('Model IDN:\t\t',my_model_idn)
print('Serial number:\t',my_serial_number)
print('Firmware:\t\t',my_firmware)

#Close connection
my_device.close()
if my_device.isOpen():
    print('\nconnection with my_device is open')
else:
    print('\nconnection with my_device is close')

```

2.1.2 readSettingsExample.py

```

# -*- coding: utf-8 -*-
"""readSettingsExample

    Created on Tue May 7 09:49 2024

    Connects to a Tausand Tempico device and reads its settings.

    Instructions:
        * before running this example, pyTempico package must be installed.
        * change 'my_port' to your corresponding port.
        * run.

    | @author: David Guzman, Tausand Electronics
    | dguzman@tausand.com
    | https://www.tausand.com
    """

import pyTempico

my_port = 'COM7' #change this port to your Tempico device's port

my_device = pyTempico.TempicoDevice(my_port) #create object

print('opening connection with device in port',my_port)
my_device.open() #open connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

print('\n1) reading general getSettings')

```

(continues on next page)

(continued from previous page)

```

my_settings = my_device.getSettings()
print('settings:')
print(my_settings)

print('\n2) reading specific settings')
print('NumberOfRuns:', my_device.getNumberOfRuns())
print('ThresholdVoltage:', my_device.getThresholdVoltage())

print('\n2.1) reading channel specific settings using direct channel method')
print('Ch1 AverageCycles:', my_device.ch1.getAverageCycles())
print('Ch2 AverageCycles:', my_device.ch2.getAverageCycles())
print('Ch3 AverageCycles:', my_device.ch3.getAverageCycles())
print('Ch4 AverageCycles:', my_device.ch4.getAverageCycles())
print('Ch1 Mode:', my_device.ch1.getMode())
print('Ch1 NumberOfStops:', my_device.ch1.getNumberOfStops())
print('Ch1 isEnabled:', my_device.ch1.isEnabled())
print('Ch1 StartEdge:', my_device.ch1.getStartEdge())
print('Ch1 StopEdge:', my_device.ch1.getStopEdge())
print('Ch1 StopMask:', my_device.ch1.getStopMask())

print('\n2.2) reading channel specific settings via device method')
print('Ch1 AverageCycles:', my_device.getAverageCycles(1))
print('Ch2 AverageCycles:', my_device.getAverageCycles(2))
print('Ch3 AverageCycles:', my_device.getAverageCycles(3))
print('Ch4 AverageCycles:', my_device.getAverageCycles(4))
print('Ch1 Mode:', my_device.getMode(1))
print('Ch1 NumberOfStops:', my_device.getNumberOfStops(1))
print('Ch1 isEnabled:', my_device.isEnabled(1))
print('Ch1 StartEdge:', my_device.getStartEdge(1))
print('Ch1 StopEdge:', my_device.getStopEdge(1))
print('Ch1 StopMask:', my_device.getStopMask(1))

print('\nclosing connection with device in port',my_port)
my_device.close()           #close connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

```

2.1.3 writeSettingsExample.py

```

# -*- coding: utf-8 -*-
"""writeSettingsExample

Created on Tue Jul 14 16:42 2025

This example connects to a Tausand Tempico device and demonstrates how to
read and modify its basic settings, including average cycles, number of stops,
and stop mask – both using direct channel access and the main device interface.

Instructions:
- Make sure the pyTempico package is installed.
- Replace 'COM5' with the appropriate serial port for your Tempico device.
- Run the script.

```

(continues on next page)

(continued from previous page)

```

    | @author: Joan Amaya, Tausand Electronics
    | jamaya@tausand.com
    | https://www.tausand.com
"""

import pyTempico

# Set your device port here
my_port = 'COM5'
my_device = pyTempico.TempicoDevice(my_port)

print(f"\nOpening connection on port {my_port}...")
my_device.open()

if my_device.isOpen():
    print("Device connection is open.")
else:
    print("Failed to open device connection.")

# Reset to default settings before making changes
print("\nResetting device to default settings...")
my_device.reset()

# Read initial settings
print("\n[1] Reading default settings with my_device.getSettings():")
settings = my_device.getSettings()
print(settings)

# Modify average cycles (Channel 1)
print("\n[2] Setting average cycles on Channel 1 using direct channel method (Channel 1 → 4 cycles):")
my_device.ch1.setAverageCycles(4)
print(f"New value: {my_device.ch1.getAverageCycles()}")

# Reset and apply the same setting using device-level method
print("\nResetting...")
my_device.reset()
print("\n[3] Setting average cycles via device method (Channel 1 → 8 cycles):")
my_device.setAverageCycles(1, 8)
print(f"Verified: {my_device.getAverageCycles(1)}")

# Modify number of stops
print("\nResetting...")
my_device.reset()
print("\n[4] Setting number of stops on Channel 1 using direct method (Channel 1 → 5 stops):")
my_device.ch1.setNumberOfStops(5)
print(f"New value: {my_device.ch1.getNumberOfStops()}")

# Reset and set using device method
print("\nResetting...")
my_device.reset()
print("\n[5] Setting number of stops via device method (Channel 1 → 4 stops):")
my_device.setNumberOfStops(1, 4)
print(f"Verified: {my_device.getNumberOfStops(1)}")

```

(continues on next page)

(continued from previous page)

```

# Modify stop mask
print("\nResetting...")
my_device.reset()
print("\n[6] Setting stop mask on 1000 us in Channel 1 using direct method (Channel 1 → 1000):")
my_device.ch1.setStopMask(1000)
print(f"New value: {my_device.ch1.getStopMask()}")

# Reset and set using device method
print("\nResetting...")
my_device.reset()
print("\n[7] Setting stop mask on 2000 us via device method (Channel 1 → 2000):")
my_device.setStopMask(1, 2000)
print(f"Verified: {my_device.getStopMask(1)}")

# Close connection
print(f"\nClosing connection on port {my_port}...")
my_device.close()

if my_device.isOpen():
    print("Connection is still open.")
else:
    print("Connection closed successfully.")

```

2.1.4 dateTimeExample.py

```

# -*- coding: utf-8 -*-
"""dateTimeExample

    This script demonstrates how to connect to a Tausand Tempico device and
    perform several operations related to time management and measurement flow.

    Instructions:
        * ensure `pyTempico` is installed,
        * change 'my_port' to your corresponding port,
        * run the script.

    You do not need a connected signal to run this example.

    This example:
        * opens connection to the device,
        * reset and configure channels,
        * read and synchronize time,
        * modifies allowed datetime range,
        * performs a measurement,
        * closes connection.

    | @author: Joan Amaya, Tausand Electronics
    | jamaya@tausand.com
    | https://www.tausand.com
"""
import pyTempico

my_port = 'COM5' #change this port to your Tempico device's port

```

(continues on next page)

(continued from previous page)

```

my_device = pyTempico.TempicoDevice(my_port)    #create object

print('opening connection with device in port',my_port)
my_device.open()                               #open connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

print('\nreseting device. This clears previous measurements, and changes settings to_
↳default values.')
my_device.reset()

#Enable channel 1, disable channels 2-4
print('\ndisabling channels 2-4.')
my_device.ch1.enableChannel()    #optional, since enabled by default
my_device.ch2.disableChannel()
my_device.ch3.disableChannel()
my_device.ch4.disableChannel()

print("\nWe want to see the current time in seconds with microsecond resolution. By_
↳default, this time is taken when the device is powered on.\n")
print(f"{my_device.getDateTime()}")

print("\nThis time can also be printed in a formatted date style, which will become_
↳more useful later on.\n")
print(f"{my_device.getDateTime(True)}")

print("\nThis time can be synchronized by the user, who provides a timestamp in_
↳seconds with microsecond resolution.\n")

print("\nHowever, this timestamp must fall within the minimum and maximum limits_
↳accepted by the device.\n")
print(f"Maximum value with my_device.getMaximumDatetime(): {my_device.
↳getMaximumDatetime()}")
print(f"Formatted (with True): {my_device.getMaximumDatetime(True)}")
print(f"Minimum value with my_device.getMinimumDatetime(): {my_device.
↳getMinimumDatetime()}")
print(f"Formatted (with True): {my_device.getMinimumDatetime(True)}")

print("\nNow that we know the allowed range, we'll configure a custom time value.\n")
print("Setting the time using: my_device.setDateTime(3102462800.0)")
my_device.setDateTime(3102462800.0)

print("\nWe can now verify that the time was correctly synchronized.\n")
print(f"Device time via my_device.getDateTime(): {my_device.getDateTime()}")

print("\nAlso we can synchronize the time with our pc, using the same function_
↳without parameter.\n")
print("Setting the time using: my_device.setDateTime()")
my_device.setDateTime()

print("\nWe can now verify that the time was correctly synchronized.\n")
print(f"Device time via my_device.getDateTime(): {my_device.getDateTime()}")
print(f"With a date format now my_device.getDateTime(True): {my_device.
↳getDateTime(True)}")

```

(continues on next page)

(continued from previous page)

```

print("\nIf we want to use a timestamp outside the current limits, we can update the_
↪allowed range.\n")
print(f"Setting new maximum with my_device.setMaximumDatetime(4103462800.0).")
my_device.setMaximumDatetime(4103462800.0)
print(f"Setting new minimum with my_device.setMinimumDatetime(1576854800.0).")
my_device.setMinimumDatetime(1576854800.0)

print("\nNow we can retrieve the new minimum and maximum values.\n")
print(f"New maximum: my_device.getMaximumDatetime(): {my_device.getMaximumDatetime()}
↪")
print(f"New minimum: my_device.getMinimumDatetime(): {my_device.getMinimumDatetime()}
↪")

print("\nThese values are useful to determine when a start signal was received. We_
↪will now perform a measurement – a connected start signal is required.\n")

print("Disabling channels 2-4.")
my_device.ch1.enableChannel()
my_device.ch2.disableChannel()
my_device.ch3.disableChannel()
my_device.ch4.disableChannel()

print("\nStarting a measurement.\n")
my_device.measure()

print("my_device.fetch():")
data = my_device.fetch()
print(data)

print("\nNote that the start time aligns with the synchronized time. Additionally, we_
↪can retrieve the time of the last start and the last synchronization (i.e., from_
↪setDateTime()).\n")
print(f"Last start via my_device.getLastStart(): {my_device.getLastStart()}")
print(f"Last start (formatted): {my_device.getLastStart(True)}")
print(f"Last synchronization via my_device.getLastSync(): {my_device.getLastSync()}")
print(f"Last synchronization (formatted): {my_device.getLastSync(True)}")

my_device.close()
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

```

2.2 Measurement examples

Examples on how to measure start-stop times with a Tausand Tempico.

2.2.1 singleMeasureExample.py

```
# -*- coding: utf-8 -*-
"""singleMeasureExample

    Created on Tue May  7 09:23 2024

    Connects to a Tausand Tempico device, starts a measurement and reads the
    results.

    Instructions:
        * before running this example, pyTempico package must be installed.
        * change 'my_port' to your corresponding port.
        * connect signals to your Tempico Device. If no signals are measured,
        this example will return an empty data array.
        * run.

    | @author: David Guzman, Tausand Electronics
    | dguzman@tausand.com
    | https://www.tausand.com
"""

import pyTempico

my_port = 'COM16' #change this port to your Tempico device's port

my_device = pyTempico.TempicoDevice(my_port)    #create object

print('opening connection with device in port',my_port)
my_device.open()    #open connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

print('sending a measure request to device')
data = my_device.measure()    #starts a measurement, and saves response in 'data'
print('measured data, in ps:',data)

my_device.close()    #close connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')
```

2.2.2 modeTwoMeasureExample.py

```
# -*- coding: utf-8 -*-
"""modeTwoMeasureExample

Created on Tue May 7 11:42 2024

Connects to a Tausand Tempico device, changes settings to measure only in
channel 1 in mode 2 (125ns-4ms mode). Then, starts a measurement and reads
the results.

Instructions:
    * before running this example, pyTempico package must be installed.
    * change 'my_port' to your corresponding port.
    * connect signals to your Tempico Device. If no signals are measured,
      this example will return an empty data array.
    * run.

/ @author: David Guzman, Tausand Electronics
/ dguzman@tausand.com
/ https://www.tausand.com
"""

import pyTempico

my_port = 'COM16' #change this port to your Tempico device's port

my_device = pyTempico.TempicoDevice(my_port)    #create object

print('opening connection with device in port',my_port)
my_device.open()                               #open connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

print('\nreseting device. This clears previous measurements, and changes settings to_
↪default values.')
my_device.reset()

#Enable channel 1, disable channels 2-4
print('\ndisabling channels 2-4.')
my_device.ch1.enableChannel()    #optional, since enabled by default
my_device.ch2.disableChannel()
my_device.ch3.disableChannel()
my_device.ch4.disableChannel()

print('\nchanging measurement mode in channel 1 to mode 2')
my_device.ch1.setMode(2)        #default mode is 1, changing to mode 2
#verify
print('my_device.ch1.getMode():',my_device.ch1.getMode())

print('\nsending a measure request to device')
data = my_device.measure()      #starts a measurement, and saves response in 'data'
print('measured data, in ps:',data)

print('fetch:',my_device.fetch()) #fetch most recent data
```

(continues on next page)

(continued from previous page)

```

my_device.close()           #close connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

```

2.2.3 numberOfRunsMeasureExample.py

```

# -*- coding: utf-8 -*-
"""numberOfRunsMeasureExample

Created on Tue Jul 14 16:42 2025

Example script for measuring with a Tausand Tempico device using extended
number of runs (1000), and using Mode 2 (125 ns - 4 ms range) on Channel 1.

This example connects to the Tempico device, configures it to only enable
Channel 1, sets the measurement mode to Mode 2, and adjusts the number of
runs (measurements) to 1000. It then starts a single measurement operation
and reads the resulting data.

The key feature demonstrated here is the use of `setNumberOfRuns(1000)`,
which tells the device to internally perform 1000 measurements in response
to a single call to `measure()`. This is ideal for bulk acquisition workflows
where multiple time intervals are collected automatically.

Instructions:
- Make sure the `pyTempico` package is installed.
- Replace `COM5` with the serial port corresponding to your device.
- Connect appropriate start/stop signals to the Tempico device.
- Run the script. If no signals are present, the measurement will return
  an empty array.

| @author: Joan Amaya, Tausand Electronics
| jamaya@tausand.com
| https://www.tausand.com
"""
import pyTempico

my_port = 'COM5' #change this port to your Tempico device's port

my_device = pyTempico.TempicoDevice(my_port)    #create object

print('opening connection with device in port',my_port)
my_device.open()           #open connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

print('\nreseting device. This clears previous measurements, and changes settings to
↳ default values.')
my_device.reset()

```

(continues on next page)

(continued from previous page)

```

#Enable channel 1, disable channels 2-4
print('\ndisabling channels 2-4.')
my_device.ch1.enableChannel()    #optional, since enabled by default
my_device.ch2.disableChannel()
my_device.ch3.disableChannel()
my_device.ch4.disableChannel()

print('\nchanging measurement mode in channel 1 to mode 2')
my_device.ch1.setMode(2)        #default mode is 1, changing to mode 2
#verify
print('my_device.ch1.getMode():',my_device.ch1.getMode())

print('\nchanging number of runs to 1000, this setting is applied to every channel in_
↳the device')
my_device.setNumberOfRuns(1000)    #default number of run is 1, changing to number of_
↳run 1000
#verify
print('my_device.getNumberOfRuns():',my_device.getNumberOfRuns())

print('\nsending a measure request to device')
data = my_device.measure()    #starts a measurement, and saves response in 'data'
print('measured data, in ps:',data)

print('fetch:',my_device.fetch()) #fetch most recent data
print('\nIf a measurement was succesful, you will see 1000 different measurements_
↳with a single call of the measurement function.')

my_device.close()
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

```

2.2.4 numberOfStopsMeasureExample.py

```

# -*- coding: utf-8 -*-
"""numberOfStopsMeasureExample

Created on Tue Jul 14 16:42 2025

Example script for measuring with a Tausand Tempico device using Mode 2
(125 ns - 4 ms range) on Channel 1, with multiple stops per measurement.

This example connects to the Tempico device, enables only Channel 1, sets
the measurement mode to Mode 2, and changes the number of stops to 5. It
then starts a measurement and prints the result.

The key feature demonstrated here is the use of `setNumberOfStops(5)`,
which configures the device to collect **5 stop events** after a single
start signal during each measurement cycle. This is useful for analyzing
multiple photon arrivals or signal repetitions after a common trigger.

Instructions:

```

(continues on next page)

(continued from previous page)

```

- Make sure the `pyTempico` package is installed.
- Replace `COM5` with the serial port corresponding to your device.
- Connect a periodic signal to the **start input**, and the signal to
  be measured to the stop input of Channel 1.
- Run the script. If no signals are received, the measurement will
  return an empty array.

/ @author: Joan Amaya, Tausand Electronics
/ jamaya@tausand.com
/ https://www.tausand.com
"""
import pyTempico

my_port = 'COM5' #change this port to your Tempico device's port

my_device = pyTempico.TempicoDevice(my_port)    #create object

print('opening connection with device in port',my_port)
my_device.open()                               #open connection with device
if my_device.isOpen():
    print('connection with my_device is open')
else:
    print('connection with my_device is close')

print('\nreseting device. This clears previous measurements, and changes settings to
↳default values.')
my_device.reset()

#Enable channel 1, disable channels 2-4
print('\ndisabling channels 2-4.')
my_device.ch1.enableChannel()    #optional, since enabled by default
my_device.ch2.disableChannel()
my_device.ch3.disableChannel()
my_device.ch4.disableChannel()

print('\nchanging measurement mode in channel 1 to mode 2')
my_device.ch1.setMode(2)        #default mode is 1, changing to mode 2
#verify
print('my_device.ch1.getMode():',my_device.ch1.getMode())

print('\nchanging number of stops in channel 1 to 5')
my_device.ch1.setNumberOfStops(5)    #default number of stops is 1, changing number
↳of stops to 5
#verify
print('my_device.ch1.getNumberOfStops():',my_device.ch1.getNumberOfStops())

print('\nsending a measure request to device')
data = my_device.measure()    #starts a measurement, and saves response in 'data'
print('measured data, in ps:',data)

print('fetch:',my_device.fetch()) #fetch most recent data
print('\nYou will see 5 stop data per measurement, these stops are captured after the
↳same start signal.')

my_device.close()
if my_device.isOpen():
    print('connection with my_device is open')

```

(continues on next page)

(continued from previous page)

```

else:
    print('connection with my_device is close')

```

2.2.5 multipleMeasureSingleChExample.py

```

# -*- coding: utf-8 -*-
"""
multipleMeasureSingleChExample

    Created on Mon Feb 16 10:35:56 2026

    Connects to a Tausand Tempico device, starts a sequence of multiple
    measurements on a single channel, and reads the results. Results are
    written in a csv file.

    Instructions:
        * before running this example, pandas and pyTempico packages must be
        installed.
        * change 'my_port' to your corresponding port.
        * (optional) change parameters 'mode', 'number_of_measurements'
        and 'number_of_runs'.
        * connect signals to your Tempico Device. If no signals are measured,
        this example will return an empty data array.
        * run.

    | @author: David Guzman, Tausand Electronics
    | dguzman@tausand.com
    | https://www.tausand.com

"""
import pyTempico
import pandas as pd
from datetime import datetime

my_port = 'COM7' #change this port to your Tempico device's port
my_device = pyTempico.TempicoDevice(my_port) #create object
mode = 1 #valid modes: 1 or 2. (TOF up to 250ns, or TOF from 125ns up to 4ms)
number_of_measurements = 50
number_of_runs = 100 #valid values: 1 to 1000

def MeasureStartStopSingleCh(selected_ch):
    try:
        selected_ch = int(selected_ch)
        if (selected_ch < 1) or (selected_ch > numch):
            selected_ch=""
            raise Exception("Invalid input.")
    except ValueError:
        selected_ch=""
        print("Invalid input.")

    selected_ch_letter = chr(ord('A')+selected_ch-1)
    print('\nPreparing to measure start-stop TOF in channel '+selected_ch_letter+'.')
    print('Connect signals into start and stop'+selected_ch_letter+'.')

```

(continues on next page)

(continued from previous page)

```

print("Are signals connected (y/n):")
user_input = input()
data = []
if (user_input == 'y') or (user_input == 'Y'):
    for i in range(1,numch+1):
        if i == selected_ch:
            #enable selected channel
            my_device.enableChannel(i)
        else:
            #disable unselected channels
            my_device.disableChannel(i)

    my_device.setMode(selected_ch, mode)
    my_device.setNumberOfRuns(number_of_runs)
    print('Measure started. Running '+str(number_of_measurements),end='')
    print(' measurements, each of '+str(number_of_runs)+' runs.')

    #synchronize Tempico's datetime with PC's datetime
    my_device.setDateTime()

    #calibrate before starting a measurement
    my_device.calibrateDelay()

    print('Progress:')
    for k in range(1,number_of_measurements+1):
        #repeat number_of_measurements times the measurment
        percentage=100.0*k/number_of_measurements
        print(f"\r{percentage:4.0f}%",end='')
        this_data = my_device.measure() #starts a measurement, and saves_
↪response in 'data'
        data = data + this_data #appending

    df = pd.DataFrame(data, columns = ["ch", "run", "datetime", "tof_ps"])
    print('\nMeasure completed.')

    print('\nTime-Of-Flight statistics:')
    samples = len(df)
    tof_mean = round(df['tof_ps'].mean(),2)
    tof_median = df['tof_ps'].median()
    tof_std = round(df['tof_ps'].std(),2)
    tof_max = df['tof_ps'].max()
    tof_min = df['tof_ps'].min()
    print(f" samples:      \t{samples:14d}")
    print(f" mean (ps):    \t{tof_mean:17.2f}")
    print(f" median (ps): \t{tof_median:14.0f}")
    print(f" std (ps):     \t{tof_std:17.2f}")
    print(f" max (ps):      \t{tof_max:14.0f}")
    print(f" min (ps):      \t{tof_min:14.0f}")

    #build csv file name
    dt_string=datetime.today().strftime('%Y%m%d%H%M')
    csv_file_name='tempico_tof_ch'+str(selected_ch)+'_'+dt_string+'.csv'

    #write header with device details into csv file
    my_header = {
        'label': ['Datetime', 'Tempico model', 'Serial number', 'Firmware', 'mode
↪'],

```

(continues on next page)

(continued from previous page)

```

        'value': [datetime.today().strftime('%Y-%m-%d %H:%M'), my_device.
↳getModelIdn(), my_device.getSerialNumber(), my_device.getFirmware(), my_device.
↳getMode(selected_ch)]
    }
    header_df = pd.DataFrame(my_header)
    header_df.to_csv(csv_file_name,header=False,index=False) #no printing column_
↳names or row indexes

    #write data statistics into csv file
    my_stats = {
        'label': ['samples','tof_ps mean','tof_ps median','tof_ps standard_
↳deviation','tof_ps maximum','tof_ps minimum'],
        'value' : [samples, tof_mean, tof_median, tof_std, tof_max, tof_min]
    }
    stats_df = pd.DataFrame(my_stats)
    stats_df.to_csv(csv_file_name,header=False,index=False,mode='a') #no printing_
↳column names or row indexes, mode 'append'

    #write dataset into file
    df.to_csv(csv_file_name,mode='a') #mode 'append'
    print('\nData saved in '+csv_file_name)

else:
    print('Measurement cancelled.')

#####      MAIN      #####

try:
    print('opening connection with device in port',my_port)
    my_device.open() #open connection with device
    if my_device.isOpen():
        print('connection with my_device is open')
    else:
        print('connection with my_device is close')
        raise Exception("Failed opening a conection in port "+my_port)

    numch = my_device.number_of_channels

    print('\nSelect a stop channel to measure (1 to '+str(numch)+'):')
    user_input = input()

    MeasureStartStopSingleCh(user_input) #call measurement routine

    my_device.close()
    if my_device.isOpen():
        print('\nconnection with my_device is open')
    else:
        print('\nconnection with my_device is close')

except Exception as e:
    print(e)

finally:
    if my_device.isOpen():
        my_device.close() #close a connection, if it is still open

```

2.3 TP1200 examples

Examples specific for Tausand Tempico TP1200 devices.

2.3.1 calibrateExample.py

```
# -*- coding: utf-8 -*-
"""calibrateExample

    Created on Thu Jan 29 11:26 2026

    This script demonstrates how to calibrate a Tausand Tempico TP1200 device.

    In TP1000 the theoretical hardware delay is 0ns. Calibration has no effect.
    In TP1200 the theoretical hardware delay is 250ns. Calibration measures the
    actual hardware delay, and adjusts the measurement values in consequence.

    Instructions:
        * before running this example, pyTempico package must be installed.
        * change 'my_port' to your corresponding port.
        * run.

    You do not need a connected signal to run this example.

    This example:
        * opens connection to the device,
        * resets the device (optional step),
        * reads current hardware delays,
        * calibrates,
        * reads updated hardware delays,
        * closes connection.

    | @author: David Guzman, Tausand Electronics
    | dguzman@tausand.com
    | https://www.tausand.com
"""
import pyTempico

my_port = 'COM7' #change this port to your Tempico device's port

my_device = pyTempico.TempicoDevice(my_port)    #create object

try:
    print('opening connection with device in port',my_port)
    my_device.open()                #open connection with device
    if my_device.isOpen():
        print('connection with my_device is open')
    else:
        print('connection with my_device is close')
        raise Exception("Failed opening a conection in port "+my_port)

    print('\n1) Reseting device. This clears previous measurements, and changes_
↪ settings to default values.')
    my_device.reset()
```

(continues on next page)

(continued from previous page)

```

#get the number of channels of the connected device
my_number_of_channels = my_device.number_of_channels

#Read current inner delay in each channel
print('\n2) Read inner start-stop hardware delay.')
for i in range(1,my_number_of_channels+1): #from 1 to 4 inclusive if TP1xx4
    print('        delay in channel',i,end='\t')
    print(my_device.getDelay(i),end='')
    print('ps')

#Get last calibration time
print(f"\nLast delay calibration via my_device.getLastDelaySync(): {my_device.
↪getLastDelaySync()}")
print(f"Last delay calibration (formatted): {my_device.getLastDelaySync(True)}")

#Read current inner delay in each channel
print('\n3) Request a calibration.')
my_device.calibrateDelay()

print('\n4) Read again inner start-stop hardware delay, after calibration.')
for i in range(1,my_number_of_channels+1): #from 1 to 4 inclusive if TP1xx4
    print('        delay in channel',i,end='\t')
    print(my_device.getDelay(i),end='')
    print('ps')

#Get last calibration time
print(f"\nLast delay calibration via my_device.getLastDelaySync(): {my_device.
↪getLastDelaySync()}")
print(f"Last delay calibration (formatted): {my_device.getLastDelaySync(True)}")

my_device.close()
if my_device.isOpen():
    print('\nconnection with my_device is open')
else:
    print('\nconnection with my_device is close')

except Exception as e:
    print(e)

finally:
    if my_device.isOpen():
        my_device.close() #close a connection, if it is still open

```

2.3.2 calibrateOffsetExample.py

```

# -*- coding: utf-8 -*-
"""
calibrateOffsetExample

Created on Mon Feb 16 10:32 2026
Last updated on Wed May 20 16:34 2026

```

(continues on next page)

(continued from previous page)

This script demonstrates how to calibrate hardware delay offset of a Tausand Tempico TP1200 device. These offsets are intended to fine tune each TP1200 device.

Changing hardware delay offset should only be done by qualified personnel.

In TP1004, the theoretical hardware delay is 0ns. No delay offset applies. In TP1204, the theoretical hardware delay is 250ns. The theoretical delay offset is 0ns.

This example:

- * opens connection to a Tempico device,*
- * launches a menu with the following options,*
 - 1. Read delay offsets*
 - 2. Read default delay offsets*
 - 3. Calibrate delay offset in single channel*
 - 4. Change hardware delay offset*
 - 0. Exit*

*You ****need**** a duplicated signal connected to start and stop to run this routine.*

Instructions:

- 1. Ensure `pyTempico` and `pandas` are installed.*
- 2. Run the script.*
- 3. Enter the communication port of Tempico device when requested (e.g. 'COM7')*

*| @author: David Guzman, Tausand Electronics
| dguzman@tausand.com
| https://www.tausand.com*

"""

```
import pyTempico
import pandas as pd
from datetime import datetime
```

```
def ReadTempicoIdentifiers():
    #Use get methods to obtain identifier strings of a Tausand Tempico device
    my_idn = my_device.getIdn()
    my_model_idn = my_device.getModelIdn()
    my_serial_number = my_device.getSerialNumber()
    my_firmware = my_device.getFirmware()
    #Print string identifiers
    print('\nIDN string:\t\t',my_idn)
    print('Model IDN:\t\t',my_model_idn)
    print('Serial number:\t',my_serial_number)
    print('Firmware:\t\t',my_firmware)

def ReadTempicoDelayOffsets():
    print('\nReading current delay offsets in each channel')
    delay_offsets = []
    for i in range(1,numch+1):
        msg='CALibrate:CH'+str(i)+':DElay:OFFSet?'
        my_device.writeMessage(msg)
        resul = my_device.readMessage()
        print("\t"+msg+" "+resul)
        delay_offsets.append(float(resul))
```

(continues on next page)

(continued from previous page)

```

    return delay_offsets

def ReadTempicoDelayOffsetDefaults():
    print('\nReading delay offset defaults in each channel')
    default_delay_offsets = []
    for i in range(1,numch+1):
        msg='CALibrate:CH'+str(i)+':DElay:OFFSet:DEfault?'
        my_device.writeMessage(msg)
        resul = my_device.readMessage()
        print("\t"+msg+" "+resul)
        default_delay_offsets.append(float(resul))
    return default_delay_offsets

def ResetAndCalibrateSingleCh(selected_ch):
    try:
        selected_ch = int(selected_ch)
        if (selected_ch < 1) or (selected_ch > numch):
            selected_ch=""
            raise Exception("Invalid input.")
    except ValueError:
        selected_ch=""
        print("Invalid input.")

    selected_ch_letter = chr(ord('A')+selected_ch-1)
    print('\nPreparing settings on channel '+selected_ch_letter)

    #reset to default settings
    my_device.reset()

    #set stopMask to its minimum
    my_device.setStopMask(selected_ch, my_device.getStopMaskMinimum())

    #set mode=1 for higher-resolution, TOF up to 250ns
    new_mode = 1
    my_device.setMode(selected_ch, new_mode)

    #calibrate inner delays before launching the measurement sequence
    my_device.calibrateDelay()

def MeasureStartStopSingleCh(selected_ch):
    try:
        selected_ch = int(selected_ch)
        if (selected_ch < 1) or (selected_ch > numch):
            selected_ch=""
            raise Exception("Invalid input.")
    except ValueError:
        selected_ch=""
        print("Invalid input.")

    selected_ch_letter = chr(ord('A')+selected_ch-1)
    print('\nMeasure start-stop in channel '+selected_ch_letter)
    print('Connect a duplicated signal into start and stop'+selected_ch_letter+' (e.g.
    ↪ 1kHz pulse)')
    user_input = input("Are signals connected? (y/n): ")

```

(continues on next page)

(continued from previous page)

```

data = []
if (user_input == 'y') or (user_input == 'Y'):
    #print('yes')
    my_device.setMode(selected_ch, 1)
    for i in range(1,numch+1):
        if i == selected_ch:
            #enable selected channel
            my_device.enableChannel(i)
        else:
            #disable unselected channels
            my_device.disableChannel(i)

    my_device.setNumberOfRuns(100)
    print('\nMeasure started. Progress:')
    for k in range(1,50+1):
        #repeat 50 times the measurment
        percentage=100*k/50
        print(f"\r{percentage:4.0f}%",end='')
        this_data = my_device.measure() #starts a measurement, and saves
→response in 'data'
        data = data + this_data #appending

    df = pd.DataFrame(data, columns = ["ch", "run", "datetime", "tof_ps"])
    print('\nMeasure completed.')

    overflow_constant = my_device.getOverflowParameter()
    df_correct = df[df['tof_ps'] != overflow_constant] #discard OVF values, (-
→1000000 in TP1204)

    print('\nTime-Of-Flight statistics:')
    samples = len(df)
    samples_ok = len(df_correct)
    samples_ovf = samples - samples_ok
    tof_mean = round(df_correct['tof_ps'].mean(),2)
    tof_median = df_correct['tof_ps'].median()
    tof_mode = df_correct['tof_ps'].mode()[0]
    tof_std = round(df_correct['tof_ps'].std(),2)
    tof_max = df_correct['tof_ps'].max()
    tof_min = df_correct['tof_ps'].min()
    print(f" samples:      \t{samples:4d}")
    print(f" ovf samples:   \t{samples_ovf:4d}")
    print(f" valid samples: \t{samples_ok:4d}")
    print(f" mean (ps):      \t{tof_mean:7.2f}")
    print(f" median (ps):    \t{tof_median:7.2f}")
    print(f" mode (ps):      \t{tof_mode:7.2f}")
    print(f" std (ps):       \t{tof_std:7.2f}")
    print(f" max (ps):       \t{tof_max:7.2f}")
    print(f" min (ps):       \t{tof_min:7.2f}")

    #Estimate a good delay offset
    print('\nEstimating recommendation for delay offset, based on measured median.
→')

    try:
        current_offset = delay_offsets[selected_ch-1]
    except:
        msg='CALibrate:CH'+str(selected_ch)+':DElay:OFFSet?'

```

(continues on next page)

(continued from previous page)

```

        my_device.writeMessage(msg)
        current_offset = float(my_device.readMessage())
        recommended_new_offset = current_offset + tof_median
        print(f' current delay offset for channel {selected_ch_letter}: \t
↪{current_offset:7.2f}')
        print(f' recommended new delay offset for channel {selected_ch_letter}: \t
↪{recommended_new_offset:7.2f}')

        #Save measured data in a csv file
        dt_string=datetime.today().strftime('%Y%m%d%H%M')
        csv_file_name='tempico_tof_ch'+str(selected_ch)+'_'+dt_string+'.csv'

        my_header = {
            'Column1': ['Datetime', 'Tempico model', 'Serial number', 'Firmware', ''],
↪ #end with blank row
            'Column2': [datetime.today().strftime('%Y-%m-%d %H:%M'), my_device.
↪getModelIdn(), my_device.getSerialNumber(), my_device.getFirmware(), ''] #end with_
↪blank row
        }
        header_df = pd.DataFrame(my_header)
        header_df.to_csv(csv_file_name,header=False,index=False) #no printing column_
↪names or row indexes
        df.to_csv(csv_file_name,mode='a') #mode 'append'
        print('\nData saved in '+csv_file_name)

    else:
        print('skip')

def ChangeTempicoDelayOffset():
    print('\nChange hardware delay offset')
    print("DANGER ZONE: Do you want to change hardware delay offset of a channel? (y/
↪n):")
    user_input = input()

    if ((user_input == 'y') or (user_input == 'Y')):
        #print('yes')

        print('Delay Offset: select channel to change (1 to '+str(numch)+')')
        user_input = input()
        try:
            selected_ch=int(user_input)
            if (selected_ch < 1) or (selected_ch > numch):
                selected_ch=""
                raise Exception("Invalid input.")
        except ValueError:
            selected_ch=""
            print("Invalid input.")

        print('Delay Offset: enter the NEW desired value, in ps')
        user_input = input()
        try:
            new_offset = float(user_input)
        except ValueError:
            new_offset = ""
            print("Invalid input.")

```

(continues on next page)

(continued from previous page)

```

    if((new_offset != "") and (selected_ch != "")):
        msg='CALibrate:DElay:OFFSet:PROtect Off'
        my_device.writeMessage(msg)
        resul = my_device.readMessage()
        print(msg+" "+resul)

        print('Change delay offset value in device:')
        msg='CALibrate:CH'+str(selected_ch)+':DElay:OFFSet '+str(new_offset)
        my_device.writeMessage(msg)
        resul = my_device.readMessage()
        print(msg+" "+resul)

        print('Read new value from device')
        msg='CALibrate:CH'+str(selected_ch)+':DElay:OFFSet?'
        my_device.writeMessage(msg)
        resul = my_device.readMessage()
        print(msg+" "+resul)

        msg='CALibrate:DElay:OFFSet:PROtect On'
        my_device.writeMessage(msg)
        resul = my_device.readMessage()
        print(msg+" "+resul)

#####      MAIN      #####

try:
    print("Write the port where the Tempico is connected (e.g. COM5).")
    my_port = input("The port is: ")
    my_device = pyTempico.TempicoDevice(my_port)      #create object

    print('\nopening connection with device in port',my_port)
    my_device.open()      #open connection with device
    if my_device.isOpen():
        print(f'connection with device in port {my_port} is open')
    else:
        print('connection with device is close')
        raise Exception("Failed opening a conection in port "+my_port)

    delay_offsets = []
    numch = my_device.number_of_channels
    ReadTempicoIdentifiers()

    while True:
        print("\nSelect the routine to execute:")
        print("1. Read delay offsets")
        print("2. Read default delay offsets")
        print("3. Calibrate delay offset in single channel")
        print("4. Change hardware delay offset")
        print("0. Exit")
        choice = input("Enter option: ")

        if choice == "0":
            break

```

(continues on next page)

(continued from previous page)

```

    if choice == "1":
        delay_offsets = ReadTempicoDelayOffsets()

    if choice == "2":
        ReadTempicoDelayOffsetDefaults()

    if choice == "3":
        print('\nSelect a stop channel to calibrate its offset (1 to '+str(numch)+
↪'):')
        user_input = input()

        if (delay_offsets == []):
            delay_offsets = ReadTempicoDelayOffsets()

        ResetAndCalibrateSingleCh(user_input)
        MeasureStartStopSingleCh(user_input)

    if choice == "4":
        ChangeTempicoDelayOffset()

my_device.close()
if my_device.isOpen():
    print(f'\nconnection with device in port {my_port} is open')
else:
    print(f'\nconnection with device in port {my_port} is close')

except Exception as e:
    print(e)

finally:
    if my_device.isOpen():
        my_device.close() #close a connection, if it is still open

```

2.4 Advanced examples

Examples specific for Tausand Tempico advanced features.

2.4.1 fcsExample.py

```

# -*- coding: utf-8 -*-
"""fcsExample

Acquires photon arrival times from a Tausand Tempico TDC device and
computes the normalized autocorrelation function  $G(\tau)$  in real time
using the multi-tau algorithm. Results are displayed in a live plot
using PySide2 and pyqtgraph.

The acquisition loop collects stop-time measurements from the Tempico
device, reconstructs a continuous relative timeline (ignoring inter-run
gaps), bins the photon arrivals into intensity bins of duration ``TAU_0``,
and feeds each bin into a ``MultiTauCorrelator`` instance. The resulting

```

(continues on next page)

(continued from previous page)

```

G(tau) curve is emitted to the GUI after every batch of runs.

Instructions:
- Make sure ``pyTempico`` and ``MultipleTauCorrelator`` are installed
  or available in the Python path.
- Replace ``MY_PORT`` with the serial port of your Tempico device
  (e.g. ``'COM4'`` on Windows or ``'/dev/ttyUSB0'`` on Linux).
- Connect a periodic signal to the start input and the photon
  detector output to the stop input of Channel 1.
- Adjust ``TAU_0``, ``NUMBER_OF_STOPS``, ``NUMBER_OF_RUNS``, and
  ``TOTAL_SECONDS`` as needed for your experiment.
- Run the script. The live G(tau) plot will appear immediately and
  update every batch of 100 runs until the acquisition finishes or
  the window is closed.

/ @author: Miguelangel García Castilo, Tausand Electronics
/ mgarcia@tausand.com
/ https://www.tausand.com
"""

import sys
import os

# Add the parent directory to the Python path so that MultipleTauCorrelator
# can be imported even when this script is run from a subdirectory
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

import time
import pyTempico
from multipleTauCorrelator import MultiTauCorrelator
import numpy as np
from PySide2.QtWidgets import QApplication, QMainWindow, QWidget, QVBoxLayout, QLabel
from PySide2.QtCore import QThread, Signal, QTimer
import pyqtgraph as pg

# ## Configuration #####
MY_PORT      = 'COM5'          # Serial port of the Tempico device
NUMBER_OF_STOPS = 2           # Stop events per run (1-5)
NUMBER_OF_RUNS  = 100         # Runs per measure() call (1-1000)
TOTAL_SECONDS   = 120         # Total acquisition duration in seconds
TAU_0          = 1_000_000    # Base bin size in picoseconds (1 ps)
PLOT_UPDATE_MS  = 500         # GUI refresh interval in milliseconds
# #####

# ## Worker thread: acquisition + correlation #####
class AcquisitionWorker(QThread):
    """
    Background thread that handles Tempico communication and multi-tau
    correlation. Runs independently of the GUI thread to keep the plot
    responsive during acquisition.

    Signals
    -----
    data_ready : Signal(object, object)
        Emitted after each measure() call with updated (taus_s, g) arrays.
    status_update : Signal(str)

```

(continues on next page)

(continued from previous page)

```

    Emitted after each measure() call with a human-readable status string.
finished_sig : Signal()
    Emitted once when the acquisition loop exits normally.
"""

# Emits (taus_s, g) arrays after every batch of runs
data_ready = Signal(object, object)
# Emits a status string for display in the GUI label
status_update = Signal(str)
# Emitted once when the acquisition loop finishes
finished_sig = Signal()

def __init__(self, port, number_of_stops, number_of_runs,
              total_seconds, tau_0_ps):
    super().__init__()
    self.port = port
    self.number_of_stops = number_of_stops
    self.number_of_runs = number_of_runs
    self.total_seconds = total_seconds
    self.tau_0_ps = tau_0_ps
    # Flag used by stop() to request a clean exit from the run() loop
    self._running = True

def stop(self):
    """Request the acquisition loop to exit on the next iteration."""
    self._running = False

def run(self):
    """
    Main acquisition and correlation loop. Runs in a separate thread.

    Connects to the Tempico device, configures Channel 1, then enters
    a loop that calls measure() repeatedly until the elapsed time exceeds
    ``total_seconds`` or ``stop()`` is called. Each batch of rows is
    processed into intensity bins and fed into the correlator. The
    updated G(tau) curve is emitted after every batch.
    """
    # --- Device connection and configuration ---
    device = pyTempico.TempicoDevice(self.port)
    device.open()
    if not device.isOpen():
        self.status_update.emit(f"ERROR: Could not open port {self.port}")
        return

    self.status_update.emit(f"Connected to {device.getIdn()}")
    # Reset clears all previous measurements and restores default settings
    device.reset()

    # Enable only Channel 1; disable the rest to avoid unwanted triggers
    device.ch1.enableChannel()
    device.ch2.disableChannel()
    device.ch3.disableChannel()
    device.ch4.disableChannel()
    # Mode 2: large measurement range (125 ns - 4 ms start-stop window)
    device.ch1.setMode(2)
    device.ch1.setNumberOfStops(self.number_of_stops)
    device.setNumberOfRuns(self.number_of_runs)

```

(continues on next page)

(continued from previous page)

```

# Instantiate the multi-tau correlator with the configured bin size
correlator = MultiTauCorrelator(tau_0=self.tau_0_ps,
                                num_levels=16, m=16)

# cursor_ps: running position on the relative timeline (ps).
# Advances by the last stop time of each run, ignoring inter-run gaps.
cursor_ps = 0
# next_bin_edge: upper boundary of the current intensity bin (ps).
# When an event crosses this boundary, the bin is closed and a new
# one starts.
next_bin_edge = self.tau_0_ps
# photons_in_bin: photon counter for the current open bin
photons_in_bin = 0

t_start = time.time() # Wall-clock time at acquisition start
call_count = 0 # Number of measure() calls completed
total_events = 0 # Total photon events collected so far

# --- Main acquisition loop ---
while self._running and (time.time() - t_start) < self.total_seconds:
    # Request one batch of (NUMBER_OF_RUNS × NUMBER_OF_STOPS) measurements
    # Each row has format: [ch, run, start_s, stop_ps1, ..., stop_psN]
    data = device.measure()

    for row in data:
        # Extract stop times in picoseconds (columns 3 onwards)
        stops_ps = row[3:]

        for t_ps in stops_ps:
            # Compute the absolute position of this event on the
            # relative timeline by adding the run's cursor offset
            abs_t = cursor_ps + t_ps

            # Close all bins that end before this event arrives.
            # Bins with zero photons (empty bins) are also fed to the
            # correlator – they carry statistical information about
            # periods of silence.
            while abs_t >= next_bin_edge:
                correlator.process_datum(photons_in_bin)
                photons_in_bin = 0
                next_bin_edge += self.tau_0_ps

            # Register this photon in the current open bin
            photons_in_bin += 1
            total_events += 1

        # Advance the timeline cursor to the last stop of this run.
        # The gap between the last stop and the next run's first stop
        # is intentionally ignored (Option B: relative timeline).
        cursor_ps += stops_ps[-1]

    call_count += 1
    elapsed = time.time() - t_start

    # Retrieve the current ACF and emit it to the GUI for plotting
    taus, g = correlator.get_correlation_curve()

```

(continues on next page)

(continued from previous page)

```

        if len(taus) > 0:
            # Convert lag times from picoseconds to seconds before emitting
            self.data_ready.emit(taus * 1e-12, g)

        self.status_update.emit(
            f"calls: {call_count} | events: {total_events} | "
            f"elapsed: {elapsed:.1f} s"
        )

        # Flush the last partially filled bin into the correlator
        correlator.process_datum(photons_in_bin)
        taus, g = correlator.get_correlation_curve()
        if len(taus) > 0:
            self.data_ready.emit(taus * 1e-12, g)

    device.close()
    self.status_update.emit("Acquisition finished.")
    self.finished_sig.emit()

# ??? Main window ?????????????????????????????????????????????????????????????????????
class MainWindow(QMainWindow):
    """
    Main application window. Contains a status label and a pyqtgraph
    PlotWidget that displays the live G(tau) autocorrelation curve.
    Starts the AcquisitionWorker thread on construction and stops it
    cleanly when the window is closed.
    """

    def __init__(self):
        super().__init__()
        self.setWindowTitle("FCS - Real-time Autocorrelation")
        self.resize(900, 550)

        # Central widget and vertical layout
        central = QWidget()
        self.setCentralWidget(central)
        layout = QVBoxLayout(central)

        # Status label: shows acquisition progress from the worker thread
        self.status_label = QLabel("Starting acquisition...")
        self.status_label.setStyleSheet("font-size: 11px; color: #444;")
        layout.addWidget(self.status_label)

        # --- pyqtgraph plot widget ---
        self.plot_widget = pg.PlotWidget()
        self.plot_widget.setBackground('w')
        self.plot_widget.setLabel('left', 'G( $\tau$ )')
        self.plot_widget.setLabel('bottom', ' $\tau$ ', units='s')
        self.plot_widget.setTitle('Autocorrelation Function - Tempico FCS')
        # Logarithmic x-axis: standard presentation for FCS curves
        self.plot_widget.setLogMode(x=True, y=False)
        self.plot_widget.showGrid(x=True, y=True, alpha=0.3)
        self.plot_widget.addLegend()

        # Horizontal dashed reference line at G = 1 (uncorrelated baseline)
        ref_line = pg.InfiniteLine(

```

(continues on next page)

(continued from previous page)

```

        pos=1.0, angle=0,
        pen=pg.mkPen('gray', width=1, style=pg.QtCore.Qt.DashLine)
    )
    self.plot_widget.addItem(ref_line)

    # Scatter plot curve: updated in real time by update_plot()
    self.curve = self.plot_widget.plot(
        [], [],
        pen=None,
        symbol='o',
        symbolSize=5,
        symbolBrush='steelblue',
        symbolPen=None,
        name='G( $\tau$ ) measured'
    )

    layout.addWidget(self.plot_widget)

    # --- Acquisition worker ---
    # Runs in a separate QThread to avoid blocking the GUI event loop
    self.worker = AcquisitionWorker(
        port = MY_PORT,
        number_of_stops = NUMBER_OF_STOPS,
        number_of_runs = NUMBER_OF_RUNS,
        total_seconds = TOTAL_SECONDS,
        tau_0_ps = TAU_0,
    )
    # Connect worker signals to GUI slots
    self.worker.data_ready.connect(self.update_plot)
    self.worker.status_update.connect(self.update_status)
    self.worker.finished_sig.connect(self.on_finished)
    self.worker.start()

def update_plot(self, taus_s, g):
    """Update the scatter plot with the latest G( $\tau$ ) values."""
    # pyqtgraph requires strictly positive x values when logMode x=True
    mask = taus_s > 0
    self.curve.setData(taus_s[mask], g[mask])

def update_status(self, msg):
    """Update the status label with the latest acquisition progress."""
    self.status_label.setText(msg)

def on_finished(self):
    """Append a completion marker to the status label."""
    self.status_label.setText(
        self.status_label.text() + "  ✓ Done."
    )

def closeEvent(self, event):
    """Stop the worker thread cleanly before closing the window."""
    # Signal the worker loop to exit, then wait for the thread to finish
    self.worker.stop()
    self.worker.wait()
    event.accept()

```

(continues on next page)

(continued from previous page)

```
# ?? Entry point ?????????????????????????????????????????????????????????????
def main():
    app = QApplication(sys.argv)
    # Enable antialiasing for smoother scatter plot markers
    pg.setConfigOptions(antialias=True)
    window = MainWindow()
    window.show()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
```

2.4.2 multipleTauCorrelator.py

```
# -*- coding: utf-8 -*-
"""MultipleTauCorrelator

    Created on [date]

    Multi-tau autocorrelation algorithm for Fluorescence Correlation
    Spectroscopy (FCS). Computes the normalized autocorrelation function
    G(tau) from a stream of intensity values (photon counts per bin) using
    the multi-tau scheme, which combines linear spacing at short lag times
    with logarithmic spacing at long lag times.

    Instructions:
    - Import this module and instantiate ``MultiTauCorrelator`` with
      the desired ``tau_0``, ``num_levels``, and ``m`` parameters.
    - Feed intensity bins one at a time using ``process_datum(value)``,
      where ``value`` is the photon count in each time bin.
    - After feeding all data, call ``get_correlation_curve()`` to
      retrieve the arrays of lag times and G(tau) values.
    - The lag times are returned in the same units as ``tau_0``.
      If ``tau_0`` is given in picoseconds, divide by 1e12 to convert
      to seconds.

    Example usage::

        correlator = MultiTauCorrelator(tau_0=100e3, num_levels=16, m=16)
        for photon_count in intensity_series:
            correlator.process_datum(photon_count)
        taus, g = correlator.get_correlation_curve()

    | @author: Miguelangel García Castillo, Tausand Electronics
    | mgarcia@tausand.com
    | https://www.tausand.com
"""

import numpy as np

# --- MULTI-TAU CORRELATOR CLASS ---
class MultiTauCorrelator:
    """
```

(continues on next page)

(continued from previous page)

Multi-tau autocorrelation algorithm for Fluorescence Correlation Spectroscopy.
 → (FCS).

Computes the normalized autocorrelation function $G(\tau)$ from a stream of intensity values (photon counts per bin) using the multi-tau scheme. This scheme uses linearly spaced channels at the base level and logarithmically spaced channels at higher levels, allowing efficient computation over many decades of lag time.

The normalization used is:

$$G(\tau) = \langle I(t) * I(t + \tau) \rangle / \langle I(t) \rangle^2$$

where $I(t)$ is the photon count in bin t .

Parameters

`tau_0` : float, optional

Base lag time (duration of the smallest bin), in any consistent time unit (e.g. picoseconds). Default is 1.0.

`num_levels` : int, optional

Number of levels in the multi-tau hierarchy. Each level k covers lag times up to $m * 2^k * \tau_0$. Default is 16.

`m` : int, optional

Number of channels (shift register length) per level. Must be a positive even integer. The first level uses all m channels with linear spacing. Higher levels use the upper $m/2$ channels to avoid overlap with the previous level. Default is 16.

Attributes

`shift_registers` : list of numpy.ndarray

Circular buffers of length m for each level, holding the m most recent binned intensity values at that level's time resolution.

`A` : list of numpy.ndarray

Accumulated cross-products for each channel at each level. Used as the numerator of $G(\tau)$.

`M_del` : list of numpy.ndarray

Accumulated sum of delayed (older) intensity values for each channel at each level. Used as part of the denominator of $G(\tau)$.

`M_dir` : numpy.ndarray

Accumulated sum of direct (current) intensity values for each level. Used as part of the denominator of $G(\tau)$.

`n_bins` : numpy.ndarray of int

Number of bins processed at each level.

`z_buffer` : numpy.ndarray

Accumulation buffer used to combine pairs of bins when passing data from level k to level $k+1$.

Examples

```
>>> correlator = MultiTauCorrelator(tau_0=100e3, num_levels=16, m=16)
```

```
>>> for photon_count in intensity_series:
```

```
...     correlator.process_datum(photon_count)
```

```
>>> taus, g = correlator.get_correlation_curve()
```

```
"""
```

```
def __init__(self, tau_0=1.0, num_levels=16, m=16):
```

(continues on next page)

(continued from previous page)

```

# Store the base lag time; all tau values will be multiples of this
self.tau_0 = tau_0
# Total number of levels in the multi-tau hierarchy
self.num_levels = num_levels
# Number of channels (shift register slots) per level
self.m = m

# One shift register per level: holds the m most recent intensity
# values at that level's time resolution. Acts as a circular buffer.
self.shift_registers = [np.zeros(m) for _ in range(num_levels)]

# A[k][i]: accumulated cross-product between current bin and the bin
# that arrived i+1 steps ago at level k. Numerator of G(tau).
self.A = [np.zeros(m) for _ in range(num_levels)]

# M_del[k][i]: accumulated sum of delayed intensity values at channel i,
# level k. Tracks the mean of the older (delayed) signal. Part of denominator.
self.M_del = [np.zeros(m) for _ in range(num_levels)]

# M_dir[k]: accumulated sum of direct (current) intensity at level k.
# Tracks the mean of the newer (direct) signal. Part of denominator.
self.M_dir = np.zeros(num_levels)

# n_bins[k]: total number of bins processed at level k.
# Used for normalization and to determine when a level has enough data.
self.n_bins = np.zeros(num_levels, dtype=int)

# z_buffer[k]: temporary accumulator that sums pairs of bins from
# level k-1 before passing them up to level k. Implements the
# factor-of-2 coarsening between levels.
self.z_buffer = np.zeros(num_levels)

def reset(self):
    """Reset all accumulators to zero (clears the correlation history)."""
    for k in range(self.num_levels):
        self.shift_registers[k][:] = 0.0
        self.A[k][:] = 0.0
        self.M_del[k][:] = 0.0
    self.M_dir[:] = 0.0
    self.n_bins[:] = 0
    self.z_buffer[:] = 0.0

def process_datum(self, value):
    """
    Feed a single intensity bin into the correlator.

    This method triggers the recursive multi-tau update starting at
    level 0. Each call corresponds to one time bin of duration ``tau_0``.

    Parameters
    -----
    value : float
        Photon count (or intensity) for the current bin. Must be
        non-negative.

    Notes
    -----

```

(continues on next page)

(continued from previous page)

```

Every two calls to this method at level k cause one new value to be
passed to level k+1 (with the two values summed), implementing the
factor-of-2 coarsening between levels.
"""
# Entry point: always start the recursive update at level 0
self._run_level(0, value)

def _run_level(self, k, z_k):
    """
    Perform the multi-tau update at level k with input value z_k.

    Updates the accumulator arrays A, M_del, and M_dir for level k,
    advances the shift register, and recursively propagates a coarsened
    value to level k+1 every two bins.

    Parameters
    -----
    k : int
        Current level index (0-based). Level 0 operates at the base
        time resolution ``tau_0``; level k operates at resolution
        ``2^k * tau_0``.
    z_k : float
        Intensity value (photon count) for the current bin at level k.

    Notes
    -----
    The update equations at each level are:

        A[k][i]      += z_k * shift_registers[k][i]    for all i
        M_del[k][i] += shift_registers[k][i]           for all i
        M_dir[k]    += z_k

    After updating, the shift register is rolled by one position and
    z_k is inserted at index 0 (most recent position).

    Every second call (when n_bins[k] is even), the accumulated
    z_buffer[k+1] is passed to _run_level(k+1) and reset to zero.
    """
    # Cross-product accumulation: multiply current bin z_k by each of the
    # m delayed values in the shift register. This is the core FCS
    # correlation operation: <I(t) * I(t + tau)> for all m lag channels.
    self.A[k] += z_k * self.shift_registers[k]

    # Accumulate the delayed values for normalization (denominator term).
    # M_del[k][i] will hold the sum of all values that have passed through
    # channel i, representing <I(t + tau)> for each lag.
    self.M_del[k] += self.shift_registers[k]

    # Accumulate the direct (current) value for normalization.
    # M_dir[k] will hold the total sum of all input values at this level,
    # representing <I(t)>.
    self.M_dir[k] += z_k

    # Count how many bins have been processed at this level
    self.n_bins[k] += 1

    # Advance the circular shift register: roll all values one position

```

(continues on next page)

(continued from previous page)

```

# to the right (older), then insert z_k at position 0 (most recent).
# After this, shift_registers[k][i] holds the value from i+1 bins ago.
self.shift_registers[k] = np.roll(self.shift_registers[k], 1)
self.shift_registers[k][0] = z_k

# Coarsening step: accumulate z_k into the buffer for the next level.
# When two consecutive bins have been accumulated (n_bins[k] is even),
# their sum is passed up to level k+1 as a single coarser bin.
# This implements the factor-of-2 time resolution reduction per level.
if k + 1 < self.num_levels:
    self.z_buffer[k+1] += z_k
    if self.n_bins[k] % 2 == 0:
        # Two bins accumulated: send their sum to the next level
        new_z = self.z_buffer[k+1]
        # Reset buffer for the next pair
        self.z_buffer[k+1] = 0
        # Recursive call: process the coarsened bin at level k+1
        self._run_level(k + 1, new_z)

def get_correlation_curve(self):
    """
    Compute and return the normalized autocorrelation function G(tau).

    Iterates over all levels and channels, skipping levels with
    insufficient data (fewer than m bins processed) and channels
    with zero delayed accumulator (to avoid division by zero).

    At level 0, all m channels are included (linear spacing).
    At levels 1 and above, only the upper m//2 channels are included
    (indices m//2 to m-1) to avoid overlap with the previous level,
    following the standard multi-tau scheme.

    The lag time for channel i at level k is:

        tau = (i + 1) * 2^k * tau_0

    The normalized correlation value is:

        G(tau) = (A[k][i] * n_bins[k]) / (M_dir[k] * M_del[k][i])

    Returns
    -----
    taus : numpy.ndarray
        Array of lag times in the same units as ``tau_0``.
    g_vals : numpy.ndarray
        Array of normalized autocorrelation values G(tau) corresponding
        to each lag time in ``taus``.

    Notes
    -----
    If fewer than ``m`` bins have been processed at a given level,
    that level is skipped and contributes no points to the output.
    This typically affects the highest levels early in an acquisition.
    """
    taus, g_vals = [], []

    for k in range(self.num_levels):

```

(continues on next page)

(continued from previous page)

```

# Skip levels that have not yet accumulated enough bins to fill
# all m channels of the shift register. Without this guard,
# the early channels would be computed from incomplete data.
if self.n_bins[k] < self.m:
    continue

# At level 0 use all m channels (linear spacing from tau_0 to m*tau_0).
# At levels k >= 1 skip the first m//2 channels because those lag
# times are already covered by the previous level at finer resolution,
# avoiding duplicate points in the output curve.
start_idx = 0 if k == 0 else self.m // 2

for i in range(start_idx, self.m):
    m_del = self.M_del[k][i]

    # Skip channels where no delayed data has been accumulated yet,
    # which would cause a division by zero in the normalization.
    if m_del == 0:
        continue

    # Normalized autocorrelation for channel i at level k:
    # G(tau) = <I(t)*I(t+tau)> / (<I(t)> * <I(t+tau)>)
    # In accumulated form:
    # numerator = A[k][i] (cross-product sum)
    # denominator = M_dir[k] / n_bins[k] * M_del[k][i] / n_bins[k]
    # Rearranging to avoid two divisions:
    # G = (A[k][i] * n_bins[k]) / (M_dir[k] * M_del[k][i])
    g = (self.A[k][i] * self.n_bins[k]) / (self.M_dir[k] * m_del)

    # Lag time for channel i at level k:
    # channel i corresponds to a delay of (i+1) bins at this level,
    # and each bin at level k has duration 2^k * tau_0.
    tau = (i + 1) * (2**k) * self.tau_0

    taus.append(tau)
    g_vals.append(g)

return np.array(taus), np.array(g_vals)

```

2.4.3 g2Example.py

```

# -*- coding: utf-8 -*-
"""g2Example

Acquires start-stop time delays from a Tausand Tempico TDC device and
computes the second-order correlation function  $g^2(\tau)$  in real time using
a normalized delay histogram. Results are displayed in a live plot using
PySide2 and pyqtgraph.

Supported devices
-----
TP1204
    Delivers signed tau values (start may arrive after stop), so the
    histogram spans the symmetric window [-WINDOW_NS, +WINDOW_NS].

```

(continues on next page)

(continued from previous page)

```

    No external delay line is required.
TP1004
    Delivers only positive tau values (start always arrives before stop).
    The histogram spans the asymmetric window [0, +WINDOW_NS].
    An external delay line is required if you need to observe tau < 0.

Instructions:
- Make sure ``pyTempico`` and ``g2Algorithm`` (G2Computer) are
  installed or available in the Python path.
- Set ``DEVICE_MODEL`` to ``'TP1204'`` or ``'TP1004'`` to match
  your hardware.
- Replace ``PORT`` with the serial port of your Tempico device
  (e.g. ``'COM4'`` on Windows or ``'/dev/ttyUSB0'`` on Linux).
- Connect both outputs of your beam splitter to the START and STOP
  inputs of the selected channel.
- Adjust ``BIN_NS``, ``WINDOW_NS``, ``N_RUNS``, and ``MODE`` as
  needed for your experiment.
- Run the script. The live  $g^2(\tau)$  plot will appear immediately and
  update after every acquisition batch until the window is closed.

| @author: Miguelangel García Castilo, Tausand Electronics
| mgarcia@tausand.com
| https://www.tausand.com
"""

# Imports
import sys
import io
import os
import csv
import time

import numpy as np

from PySide2.QtCore import Qt, QThread, Signal, Slot
from PySide2.QtWidgets import QApplication, QMainWindow

import pyqtgraph as pg

import pyTempico

from g2Algorithm import G2Computer

# Configuration
# Device
DEVICE_MODEL = "TP1204"      # 'TP1204' (signed tau) or 'TP1004' (positive tau only)
PORT          = "COM5"      # Serial port where the Tempico device is connected
N_RUNS        = 20          # Number of runs per acquisition batch

# Channel
CHANNEL       = 1            # TDC channel: 1=A | 2=B | 3=C | 4=D
CHANNEL_LABEL = chr(ord('A') + CHANNEL - 1) # Derived label: 'A', 'B', 'C', or 'D'

# TDC mode (TP1204 only; the TP1004 has a fixed range):
# 1 = range -200 ns ... +300 ns (recommended for HBT / single photons)
# 2 = range -50 ns ... +4 ms (for coincidences with larger delays)

```

(continues on next page)

(continued from previous page)

```

MODE = 1

# Histogram
BIN_NS      = 15.0      # Bin width in nanoseconds
WINDOW_NS   = 180.0     # Half-window (ns): histogram spans [-W, +W] for TP1204
                        # or [0, +W] for TP1004

# Signal source
# 'external' -> physical connector on the Tempico device
# 'internal' -> Tempico internal generator (TP12 series only)
START_SRC    = "external"
STOP_SRC     = "external"
INT_FREQ_HZ  = 100_000   # Hz -- only used when START_SRC == 'internal'

# TP1204 / TP1004 sentinel and threshold values
OVERFLOW_PARAMETER      = -1_000_000 # Returned by pyTempico when no valid stop
OUT_OF_RANGE_THRESHOLD_PS = 800_000   # 800 ns in ps; triggers a warning if exceeded
# ?????????????????????????????????????????????????????????????????????????????????????

# ?? Worker thread: device acquisition ?????????????????????????????????????????????
class HBTWorker(QThread):
    """
    Background thread that handles Tempico communication and  $g^2(\tau)$ 
    histogram accumulation. Runs independently of the GUI thread to keep
    the plot responsive during acquisition.

    The active TDC channel is selected by the CHANNEL constant above.
    Device-specific behaviour (signed vs. positive-only tau) is controlled
    by the DEVICE_MODEL constant:
        - TP1204: signed tau, symmetric histogram window.
        - TP1004: positive tau only, asymmetric histogram window.

    Signals
    -----
    data_ready : Signal(object, object, int, float, float)
        Emitted after each batch with (centres, g2, n_events,
        rate_start, rate_stop).
    status_update : Signal(str)
        Emitted after each batch with a human-readable status string.
    color_value : Signal(int)
        Severity indicator emitted with every status update:
        1 = OK, 2 = error, 3 = warning.
    dialog_signal : Signal(str)
        Emitted when more than 60% of measurements on a channel are
        out of range, carrying the channel label as a string.
    model_ready : Signal(str)
        Emitted once after connection with the device model identifier
        returned by device.getModelIdn().
    finished_sig : Signal()
        Emitted once when the acquisition loop exits normally.
    """

    data_ready      = Signal(object, object, int, float, float)
    status_update   = Signal(str)
    color_value     = Signal(int)
    dialog_signal   = Signal(str)

```

(continues on next page)

(continued from previous page)

```

model_ready    = Signal(str)
finished_sig    = Signal()

def __init__(self):
    super().__init__()

    self._running = True

    # -- Out-of-range / dialog state -----
    self.outOfRange = 0
    self.sentinelDialog = False

    # Failure-control counters (ThreadStartStop pattern)
    self.channelsNM = []
    self.noMeasurementsSequent = 0
    self.noAbortsSequent = 0
    self.consecutiveErrors = 0
    self.openDialog = False

    # -- G2Computer: owns the histogram and  $g^2(\tau)$  math -----
    # positive_only=True for TP1004 (no negative tau values)
    positive_only = (DEVICE_MODEL.upper() == "TP1004")
    self.g2 = G2Computer(BIN_NS, WINDOW_NS, N_RUNS,
                        positive_only=positive_only)

    self.t_start = None
    self.device = None

    # Saved device settings (restored after a hardware reset)
    self.numberRunsSetting = N_RUNS
    self.modeCh = MODE
    self.numberStopsCh = 1

def stop(self):
    """Request the acquisition loop to exit on the next iteration."""
    self._running = False

@Slot()
def dialogClosed(self):
    """Acknowledge the out-of-range dialog so the worker can continue."""
    self.openDialog = False

# ?? run ?????????????????????????????????????????????????????????????????????????????????????
def run(self):
    """
    Main acquisition loop. Runs in a separate thread.

    Connects to the Tempico device, configures the selected channel,
    then calls getMeasurements() in a tight loop until stop() is called.
    Emits finished_sig when the loop exits.
    """
    self.device = pyTempico.TempicoDevice(PORT)
    self.device.open()

    if not self.device.isOpen():
        self.status_update.emit(f"ERROR: could not open {PORT}")
        self.color_value.emit(2)

```

(continues on next page)

(continued from previous page)

```

        self.finished_sig.emit()
        return

model = self.device.getModelIdn()
self.model_ready.emit(model)
self.status_update.emit(f"Connected: {self.device.getIdn()}")
self.device.reset()

# Enable only the selected channel; disable the rest
for _n in (1, 2, 3, 4):
    _ch_obj = getattr(self.device, f'ch{_n}')
    if _n == CHANNEL:
        _ch_obj.enableChannel()
    else:
        _ch_obj.disableChannel()

_active_ch = getattr(self.device, f'ch{CHANNEL}')

# Mode is settable on the TP1204; the TP1004 has a fixed range
if DEVICE_MODEL.upper() == "TP1204":
    _active_ch.setMode(MODE)

_active_ch.setNumberOfStops(1)
_active_ch.setAverageCycles(1)
_active_ch.setStopMask(-0.25)          # Minimum mask on TP12 series
self.device.setNumberOfRuns(N_RUNS)

# Signal source configuration
if START_SRC == "internal":
    self.device.setGeneratorFrequency(INT_FREQ_HZ)
    self.device.setStartInternalSource(1)
else:
    self.device.setStartExternalSource(1)

if STOP_SRC == "internal":
    self.device.setStopInternalSource(1)
else:
    self.device.setStopExternalSource(1)

# Save current settings so they can be restored after a device reset
self._saveCurrentSettings()

self.t_start = time.time()

# --- Main acquisition loop ---
while self._running:
    self.getMeasurements()
    time.sleep(0.05)

# Clean shutdown: re-enable all channels before closing
try:
    for _n in (1, 2, 3, 4):
        getattr(self.device, f'ch{_n}').enableChannel()
    self.device.close()
except Exception:
    pass

```

(continues on next page)

(continued from previous page)

```

t_elapsed = time.time() - self.t_start if self.t_start else 0
self.status_update.emit(
    f"Acquisition finished. "
    f"Events: {self.g2.total_events} | Time: {t_elapsed:.1f} s"
)
self.finished_sig.emit()

# ?? getMeasurements ?????????????????????????????????????????????????????????????????
def getMeasurements(self):
    """
    Acquire one batch from the Tempico device and feed valid tau values
    to G2Computer.

    Row format returned by pyTempico:
        [ch, run, start_time_us, stop_ps1]

    Conversion:
        tau [ns] = stop_ps1 / 1000

    For the TP1204 the sign is preserved (negative tau is native; no
    external delay line needed). For the TP1004 all values are positive
    by design; negative results indicate noise and are discarded by
    G2Computer.accumulate() via the positive_only flag.

    Out-of-range events (|stop_ps1| > OUT_OF_RANGE_THRESHOLD_PS) are
    counted and trigger a warning dialog if they exceed 60% of all
    events, but are still passed to the histogram.
    """
    taus_batch_ns = []

    try:
        # Capture stdout to suppress pyTempico console output
        originalConsole = sys.stdout
        sys.stdout = io.StringIO()
        measurement = self.device.measure()
        printedOutput = sys.stdout.getvalue()
        sys.stdout = originalConsole

        # If a timeout was reported, keep fetching until the response is stable
        if "Timeout reached" in printedOutput:
            finishedMeasurement = False
            while not finishedMeasurement:
                time.sleep(1)
                newFetch = self.device.fetch()
                if newFetch == measurement:
                    finishedMeasurement = True
                    measurement = newFetch
                else:
                    measurement = newFetch

        # Handle empty measurement (ThreadStartStop pattern)
        if not measurement and self.noMeasurementsSequent < 3:
            self.noMeasurementsSequent += 1
        elif not measurement and self.noAbortsSequent >= 10:
            # Too many consecutive failures: reset and restore settings
            self.device.reset()
            self._applyCurrentSettings()

```

(continues on next page)

(continued from previous page)

```

        QThread.msleep(20)
    elif not measurement and self.noMeasurementsSequent >= 3:
        self.noAbortsSequent += 1
        self.device.abort()

    # --- Process each row in the batch ---
    if measurement:
        self.noAbortsSequent = 0
        self.noMeasurementsSequent = 0

        for run_row in measurement:
            if not run_row:
                # Empty row means no START signal was detected this run
                if "Start" not in self.channelsNM:
                    self.channelsNM = ["Start"]
                continue

            ch = run_row[0]
            if ch != CHANNEL:
                continue # Ignore data from non-selected channels

            if "Start" in self.channelsNM:
                self.channelsNM.remove("Start")

            # Number of valid stops in this row
            n_stops_row = self._getRange(run_row, self.numberStopsCh)

            for i in range(n_stops_row):
                raw_ps = run_row[3 + i] # Stop time in picoseconds

                # Discard the overflow sentinel returned by pyTempico
                if raw_ps == OVERFLOW_PARAMETER:
                    _ch_label = f"Ch{CHANNEL}"
                    if _ch_label not in self.channelsNM:
                        self.channelsNM.append(_ch_label)
                    continue

                _ch_label = f"Ch{CHANNEL}"
                if _ch_label in self.channelsNM:
                    self.channelsNM.remove(_ch_label)

            self.g2.n_starts += 1

            # Convert ps → ns (sign preserved for TP1204;
            # TP1004 only delivers positive values)
            tau_ns = raw_ps / 1e3

            # Out-of-range detection (warning only; not discarded)
            if abs(raw_ps) > OUT_OF_RANGE_THRESHOLD_PS:
                self.outOfRange += 1
                if (self.g2.n_starts > 10 and
                    (self.outOfRange / self.g2.n_starts) > 0.6 and
                    not self.sentinelDialog):
                    self.sentinelDialog = True
                    self.openDialog = True
                    self.dialog_signal.emit(f"channel {CHANNEL_LABEL}")
                    # Block until the user acknowledges the warning

```

(continues on next page)

(continued from previous page)

```

        while self.openDialog:
            time.sleep(1)
            # Re-read mode in case the user changed it
            if DEVICE_MODEL.upper() == "TP1204":
                self.modeCh = getattr(
                    self.device, f'ch{CHANNEL}')
                .getMode()

            self.g2.n_stops      += 1
            self.g2.total_events += 1

            if self.g2.isBatched:
                taus_batch_ns.append(tau_ns)
            else:
                self.g2.accumulate([tau_ns])

        else:
            if "Start" not in self.channelsNM:
                self.channelsNM = ["Start"]

            # End of batch: flush buffered taus and compute g2(τ)
            if self.g2.isBatched and taus_batch_ns:
                self.g2.accumulate(taus_batch_ns)

            t_elapsed = time.time() - self.t_start if self.t_start else 1e-9
            g2, rate_start, rate_stop, baseline = self.g2.compute_g2(t_elapsed)
            self.data_ready.emit(
                self.g2.centres, g2, self.g2.total_events, rate_start, rate_stop
            )

            # Emit status and color severity
            if self.channelsNM:
                self.color_value.emit(3)
                self.status_update.emit(
                    "No signal on: " + ", ".join(self.channelsNM)
                )
            else:
                self.color_value.emit(1)
                self.status_update.emit(
                    f"Measuring | events: {self.g2.total_events} | "
                    f"t: {t_elapsed:.1f} s | "
                    f"R_start: {rate_start:.0f} cps | "
                    f"R_stop: {rate_stop:.0f} cps | "
                    f"baseline: {baseline:.1f} counts/bin"
                )

            self.consecutiveErrors = 0

        except Exception as e:
            sys.stdout = sys.__stdout__
            self.consecutiveErrors += 1
            if self.consecutiveErrors > 10:
                self.stop()
            self.status_update.emit(f"Error: {e}")
            self.color_value.emit(2)

```

```
# ?? helpers ?????????????????????????????????????????????????????????????????????????????????????
```

(continues on next page)

(continued from previous page)

```

def _getRange(self, run, stop_number):
    """
    Return the number of valid stops present in a measurement row.

    Row format: [ch, run, start_time_us, stop_ps1, ...].
    Columns 0-2 are metadata; stop values start at index 3.
    """
    total_len = len(run)
    if total_len >= 4:
        return min(total_len - 3, stop_number)
    return 0

def _saveCurrentSettings(self):
    """Read and store the current device settings for later restoration."""
    _ch = getattr(self.device, f'ch{CHANNEL}')
    self.numberRunsSetting = self.device.getNumberOfRuns()
    self.numberStopsCh = _ch.getNumberOfStops()
    if DEVICE_MODEL.upper() == "TP1204":
        self.modeCh = _ch.getMode()

def _applyCurrentSettings(self):
    """Restore previously saved device settings after a hardware reset."""
    _ch = getattr(self.device, f'ch{CHANNEL}')
    self.device.setNumberOfRuns(self.numberRunsSetting)
    _ch.setNumberOfStops(self.numberStopsCh)
    if DEVICE_MODEL.upper() == "TP1204":
        _ch.setMode(self.modeCh)
    for _n in (1, 2, 3, 4):
        _ch_obj = getattr(self.device, f'ch{_n}')
        if _n == CHANNEL:
            _ch_obj.enableChannel()
        else:
            _ch_obj.disableChannel()

# ??? Plot window ?????????????????????????????????????????????????????????????????????????????????
class HBTPlotWindow(QMainWindow):
    """
    Minimal real-time plot window for  $g^2(\tau)$ .

    Contains nothing but a pyqtgraph PlotWidget: no buttons, no status
    bar, no spin boxes. The histogram curve is refreshed every time the
    worker thread emits new data through the data_ready signal.
    Saves  $\tau/g^2(\tau)$  data to a CSV file when the window is closed.
    """

    def __init__(self):
        super().__init__()
        self.setWindowTitle(
            f" $g^2(\tau)$  HBT -- Tausand Tempico {DEVICE_MODEL}"
            f" (channel {CHANNEL_LABEL})"
        )
        self.resize(1000, 650)

        pg.setConfigOptions(antialias=True)

        self.plot = pg.PlotWidget()

```

(continues on next page)

(continued from previous page)

```

self.plot.setBackground('w')
self.plot.setLabel('left', 'g\u207d\u00b2\u207e(\u03c4)',
                    color='#222222', size='12pt')
self.plot.setLabel('bottom', 'Delay \u03c4 (ns)',
                    color='#222222', size='12pt')
self.plot.setTitle(
    f'Second-order correlation function g\u207d\u00b2\u207e(\u03c4)'
    f' -- Tausand Tempico {DEVICE_MODEL}',
    color='#111111', size='11pt'
)
self.plot.showGrid(x=True, y=True, alpha=0.35)
self.plot.addLegend(offset=(10, 10))

# Reference lines
self.plot.addItem(pg.InfiniteLine(
    pos=1.0, angle=0,
    pen=pg.mkPen('#555555', width=1.4, style=Qt.DashLine),
    label='g\u00b2=1',
    labelOpts={'color': '#555555', 'position': 0.05}
))
self.plot.addItem(pg.InfiniteLine(
    pos=0.5, angle=0,
    pen=pg.mkPen('#aaaaaa', width=1.0, style=Qt.DotLine),
    label='g\u00b2=0.5',
    labelOpts={'color': '#aaaaaa', 'position': 0.08}
))
# tau = 0 vertical line (only meaningful for the symmetric TP1204 window)
if DEVICE_MODEL.upper() == "TP1204":
    self.plot.addItem(pg.InfiniteLine(
        pos=0.0, angle=90,
        pen=pg.mkPen('#e74c3c', width=1.0, style=Qt.DashLine),
        label='\u03c4=0',
        labelOpts={'color': '#e74c3c', 'position': 0.95}
    ))

# Step-histogram curve for g\u00b2(\u03c4)
self.curve = self.plot.plot(
    [], [],
    stepMode='center',
    fillLevel=0,
    brush=pg.mkBrush(41, 128, 185, 100),
    pen=pg.mkPen('#2980b9', width=1.5),
    name=f'g\u00b2(\u03c4) -- {DEVICE_MODEL} channel {CHANNEL_LABEL}'
)

self.setCentralWidget(self.plot)

# Reference to the worker thread (assigned from main) and last data
# buffers used for CSV export when the window is closed
self.worker = None
self.last_centres = np.array([])
self.last_g2 = np.array([])

# ?? Slots ?????????????????????????????????????????????????????????????????????
@Slot(str)
def update_model_title(self, model_str):
    """

```

(continues on next page)

(continued from previous page)

```

    Connected to HBTWorker.model_ready. Replaces the placeholder device
    name with the actual model identifier returned by device.getModelIdn(),
    both in the window title and in the plot title.
    """
    self.setWindowTitle(
        f"g\u00b2(\u03c4) HBT -- {model_str} (channel {CHANNEL_LABEL}) "
    )
    self.plot.setTitle(
        f"Second-order correlation function  g\u207d\u00b2\u207e(\u03c4) "
        f"-- {model_str}",
        color='#111111', size='11pt'
    )

@Slot(object, object, int, float, float)
def update_plot(self, centres, g2, n_events, rate_s, rate_p):
    """
    Redraw the histogram curve every time HBTWorker emits new data.

    stepMode='center' in pyqtgraph requires n+1 edges for n bin values,
    built as:
        edges = [centres[0] - bin/2, centres[1] + bin/2, ...,
                 centres[-1] + bin/2]
    """
    if len(centres) == 0 or len(g2) == 0:
        return

    centres = np.asarray(centres)
    g2       = np.asarray(g2)

    bin_ns = float(centres[1] - centres[0]) if len(centres) > 1 else BIN_NS
    edges = np.concatenate([[centres[0] - bin_ns / 2], centres + bin_ns / 2])
    self.curve.setData(edges, g2)

    # Save the latest tau/g²(τ) arrays for CSV export on close
    self.last_centres = centres
    self.last_g2      = g2

    # g²(0): bin closest to tau = 0
    z_idx = int(np.argmin(np.abs(centres)))
    g2_zero = float(g2[z_idx])

    # Auto Y-range with 15% margin
    g2_max = max(float(np.max(g2)) * 1.15, 1.5) if len(g2) else 1.5
    self.plot.setYRange(0, g2_max)

    # Compact statistics in the window title
    self.setWindowTitle(
        f"g\u00b2(\u03c4) HBT -- g2(0)={g2_zero:.3f} | "
        f"events={n_events:}, | "
        f"R_start={rate_s:.0f} cps | R_stop={rate_p:.0f} cps"
    )

def closeEvent(self, event):
    """
    Called automatically when the user clicks the close button (X).

    1. Saves the latest tau/g²(τ) data to a CSV file.

```

(continues on next page)

(continued from previous page)

```

2. Stops the HBTWorker thread cleanly.
3. Allows the window to close normally.
"""
self._save_g2_data()

if self.worker is not None and self.worker.isRunning():
    self.worker.stop()
    self.worker.wait(3000) # Wait up to 3 s for the thread to finish

super().closeEvent(event)

def _save_g2_data(self):
    """Write the latest tau_ns / g^2(τ) columns to a timestamped CSV file."""
    if self.last_centres is None or len(self.last_centres) == 0:
        print("\n[INFO] No g^2(τ) data to save yet.")
        return

    timestamp = time.strftime("%Y%m%d_%H%M%S")
    filename = f"g2_data_{timestamp}.csv"
    filepath = os.path.abspath(filename)

    with open(filepath, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["tau_ns", "g2"])
        for tau, g2_val in zip(self.last_centres, self.last_g2):
            writer.writerow([f"{tau:.6f}", f"{g2_val:.6f}"])

    print(f"\n[INFO] g^2(τ) data saved to: {filepath}")

# Signal handlers (terminal output)
@Slot(object, object, int, float, float)
def on_data_ready(centres, g2, n_events, rate_s, rate_p):
    """Print g^2(0) and basic statistics each time a batch is processed."""
    if len(centres) == 0 or len(g2) == 0:
        return
    z_idx = int(np.argmin(np.abs(centres)))
    g2_zero = float(g2[z_idx])
    print(
        f"  events: {n_events:>8,} | "
        f"g2(0): {g2_zero:.4f} | "
        f"R_start: {rate_s:,.0f} cps | "
        f"R_stop: {rate_p:,.0f} cps"
    )

@Slot(str)
def on_status(msg):
    """Print status messages emitted by the worker."""
    print(f"[status] {msg}")

@Slot(int)
def on_color(code):
    """Print a severity indicator: 1=OK, 2=ERROR, 3=WARNING."""
    labels = {1: "OK", 2: "ERROR", 3: "WARNING"}
    print(f"[{labels.get(code, '?')}]")

@Slot(str)

```

(continues on next page)

(continued from previous page)

```

def on_dialog(channel_name):
    """
    Called when more than 60% of measurements on a channel are out of range.
    Prints a warning and acknowledges immediately so the worker can continue.
    """
    print(
        f"\n[WARNING] More than 60% of measurements on {channel_name} "
        "are out of range.\n"
        "Consider switching TDC mode (1 ↔ 2) or adjusting the signal "
        "time window.\n"
    )
    # Acknowledge so the worker thread is not blocked waiting for a dialog
    worker.dialogClosed()

@Slot()
def on_finished():
    """Called when the worker finishes; exits the Qt event loop."""
    print("\nAcquisition finished. Exiting.")
    QApplication.instance().quit()

# Entry point
if __name__ == "__main__":
    app = QApplication(sys.argv)

    # Plot window: only the  $g^2(\tau)$  histogram, no control panel
    window = HBTPlotWindow()
    window.show()

    worker = HBTWorker()
    window.worker = worker # Reference stored so closeEvent can stop the thread

    # Connect worker signals to the terminal handlers and the live plot
    worker.data_ready.connect(on_data_ready)
    worker.data_ready.connect(window.update_plot)
    worker.status_update.connect(on_status)
    worker.color_value.connect(on_color)
    worker.dialog_signal.connect(on_dialog)
    worker.model_ready.connect(window.update_model_title)
    worker.finished_sig.connect(on_finished)

    print(f"Starting HBT acquisition ({DEVICE_MODEL}) -- press Ctrl+C to stop.\n")
    worker.start()

    try:
        sys.exit(app.exec_())
    except KeyboardInterrupt:
        print("\nInterrupted by user.")
        worker.stop()
        worker.wait(3000)
        sys.exit(0)

```

2.4.4 g2Algorithm.py

```
# -*- coding: utf-8 -*-
"""g2Algorithm

Pure math/signal-processing core for the real-time HBT
(Hanbury Brown and Twiss) experiment using Tausand Tempico devices
(TP1204 and TP1004) -- Second-order correlation function  $g^2(\tau)$ .

This module contains ONLY the histogram accumulation and  $g^2(\tau)$ 
normalization logic. Everything related to device communication,
channel configuration, and acquisition lives in g2Example.py.

 $g^2(\tau)$  normalization -- average of all non-empty histogram bins:
    baseline = mean( C( $\tau$ ) for all non-empty bins )
     $g^2(\tau)$  = C( $\tau$ ) / baseline

This normalization is correct when START and STOP come from the same
TDC (n_stops  $\approx$  n_starts). The classic formula  $r \cdot r \cdot T \cdot dt$  produces a
baseline of the order  $10^{-5}$ , inflating  $g^2(\tau)$  into the thousands.

Negative tau delays (start arrives after the stop) are native to the
TP1204 in mode 1 or mode 2 and do not require an external delay line.
The TP1004 only delivers positive tau values; its histogram window is
asymmetric: [0, +window_ns].

REQUIRED LIBRARIES (install before running):
    pip install numpy

| @author: Miguelangel García Castilo, Tausand Electronics
| mgarcia@tausand.com
| https://www.tausand.com
"""

# Imports
import numpy as np

# G2Computer
class G2Computer:
    """
    Stateful histogram accumulator and  $g^2(\tau)$  normalizer.

    Owns the bin edges, bin centres, and accumulated counts array.
    Receives raw tau values [ns] via accumulate() and returns the
    normalized  $g^2(\tau)$  curve via compute_g2().

    No device, no Qt, no Tempico specifics -- pure numpy math only.

    Parameters
    -----
    bin_ns : float
        Histogram bin width in nanoseconds.
    window_ns : float
        Half-width of the histogram window in nanoseconds.
        - TP1204: histogram spans [-window_ns, +window_ns].
        - TP1004: histogram spans [0, +window_ns] (positive tau only).
    """
```

(continues on next page)

(continued from previous page)

```

n_runs : int
    Number of runs per acquisition batch (used to select batched mode).
positive_only : bool, optional
    Set to True for the TP1004, which does not produce negative tau
    values. When True the histogram spans [0, +window_ns] instead of
    the symmetric [-window_ns, +window_ns] (default: False).
"""

def __init__(self, bin_ns, window_ns, n_runs, positive_only=False):
    self.bin_ns = bin_ns
    self.window_ns = window_ns
    self.positive_only = positive_only

    # -- Build histogram edges -----
    # Edges are shifted by half a bin so bin centres fall on multiples of
    # bin_ns. For the TP1204 the centre at tau = 0 is always present.
    # For the TP1004 the leftmost centre is at +bin_ns/2.
    n_half = max(int(np.ceil(window_ns / bin_ns)), 1)
    half_bin = bin_ns / 2.0

    if positive_only:
        # Asymmetric window: [0, +window_ns]
        tau_min = 0.0
    else:
        # Symmetric window: [-window_ns, +window_ns]
        tau_min = -window_ns

    edges = np.arange(0, n_half + 1) * bin_ns - half_bin + tau_min + half_bin
    # Ensure the first edge starts at tau_min
    edges = np.concatenate([[tau_min], edges[edges > tau_min]])
    # Build uniform edges spanning the full window
    n_bins_full = int(np.round((window_ns - tau_min) / bin_ns))
    if positive_only:
        edges = np.linspace(0.0, window_ns, n_bins_full + 1)
    else:
        edges = np.linspace(-window_ns, window_ns, 2 * n_half + 1)

    self.edges = edges
    self.n_bins = len(self.edges) - 1
    self.centres = 0.5 * (self.edges[:-1] + self.edges[1:])
    self.counts = np.zeros(self.n_bins, dtype=np.float64)

    # Batched mode is used when n_runs > 50 (ThreadStartStop pattern)
    self.isBatched = (n_runs > 50)

    # -- Event counters (updated by g2Example.py after each raw event) ---
    self.n_starts = 0
    self.n_stops = 0
    self.total_events = 0

    # ?? accumulate ?????????????????????????????????????????????????????????????
    def accumulate(self, taus_ns):
        """
        Add tau delays [ns] to the accumulated histogram.

        Uses floor((tau - tau_min) / bin_ns) to map the measurement window
        to indices [0, n_bins-1]. Values outside the window are silently

```

(continues on next page)

(continued from previous page)

```

discarded.

Parameters
-----
taus_ns : array-like
    Tau delay values in nanoseconds. May include negative values for
    the TP1204 (positive_only=False); negative values are silently
    discarded for the TP1004 (positive_only=True).
"""
arr = np.asarray(taus_ns, dtype=np.float64)
tau_min = 0.0 if self.positive_only else -self.window_ns
tau_max = self.window_ns

# Keep only tau values inside the measurement window
valid = (arr >= tau_min) & (arr <= tau_max)
arr = arr[valid]
if len(arr) == 0:
    return

idx = np.floor((arr - tau_min) / self.bin_ns).astype(int)
idx = np.clip(idx, 0, self.n_bins - 1)
np.add.at(self.counts, idx, 1)

# ?? compute_g2 ?????????????????????????????????????????????????????????????????
def compute_g2(self, t_elapsed):
    """
    Normalize the histogram to obtain  $g^2(\tau)$ .

    The TP1204 and TP1004 measure start→stop with a single channel:
    each start produces exactly one stop, so  $n_{\text{stops}} \approx n_{\text{starts}}$ .
    The classic formula  $\text{baseline} = r_s \cdot r_p \cdot T \cdot dt$  gives an
    extremely small baseline ( $\sim 10^{-5}$  at  $\sim 700$  cps /  $0.1$  ns bin),
    inflating  $g^2(\tau)$  into the thousands.

    The correct normalization divides by the average value of all
    non-empty histogram bins, where the correlation is flat (random-
    coincidence background):

        baseline = mean( counts[ all non-empty bins ] )
         $g^2(\tau) = \text{counts}(\tau) / \text{baseline}$ 

    If fewer than 2 non-empty bins are present, a fallback baseline of
    1.0 is used to avoid division by zero.

    Parameters
    -----
    t_elapsed : float
        Wall-clock time elapsed since acquisition started, in seconds.

    Returns
    -----
    g2 : ndarray, shape (n_bins,)
        Normalized second-order correlation function.
    rate_s : float
        Start rate in counts/s.
    rate_p : float
        Stop rate in counts/s.

```

(continues on next page)

(continued from previous page)

```
baseline : float
    Mean counts per bin used for normalization.
    """
    g2      = np.zeros(self.n_bins, dtype=np.float64)
    rate_s  = self.n_starts / t_elapsed if t_elapsed > 0 else 0.0
    rate_p  = self.n_stops   / t_elapsed if t_elapsed > 0 else 0.0

    if self.n_stops < 10:
        return g2, rate_s, rate_p, 0.0

    # Baseline = mean of ALL non-empty bins
    all_nonzero = self.counts[self.counts > 0]
    if len(all_nonzero) >= 2:
        baseline = float(np.mean(all_nonzero))
    else:
        baseline = 1.0

    if baseline > 0:
        g2 = self.counts / baseline

    return g2, rate_s, rate_p, baseline
```


INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

p

`pyTempico.core`, 3

A

`abort()` (*pyTempico.core.TempicoDevice* method), 5
`average_cycles` (*pyTempico.core.TempicoChannel* attribute), 28

C

`calibrateDelay()` (*pyTempico.core.TempicoDevice* method), 5
`ch1` (*pyTempico.core.TempicoDevice* attribute), 5
`ch2` (*pyTempico.core.TempicoDevice* attribute), 5
`ch3` (*pyTempico.core.TempicoDevice* attribute), 5
`ch4` (*pyTempico.core.TempicoDevice* attribute), 5
`channel_number` (*pyTempico.core.TempicoChannel* attribute), 28
`cleanNumberList()` (*pyTempico.core.TempicoDevice* method), 6
`close()` (*pyTempico.core.TempicoDevice* method), 6
`closeTempico()` (*pyTempico.core.TempicoDevice* method), 6
`convertReadDataToFloatList()` (*pyTempico.core.TempicoDevice* method), 6
`convertReadDataToNumberList()` (*pyTempico.core.TempicoDevice* method), 7

D

`decrementGeneratorFrequency()` (*pyTempico.core.TempicoDevice* method), 7
`decrementThresholdVoltage()` (*pyTempico.core.TempicoDevice* method), 7
`device` (*pyTempico.core.TempicoDevice* attribute), 5
`disableChannel()` (*pyTempico.core.TempicoChannel* method), 28
`disableChannel()` (*pyTempico.core.TempicoDevice* method), 8

E

`enable` (*pyTempico.core.TempicoChannel* attribute), 28
`enableChannel()` (*pyTempico.core.TempicoChannel* method), 29
`enableChannel()` (*pyTempico.core.TempicoDevice* method), 8

F

`fetch()` (*pyTempico.core.TempicoDevice* method), 8
`findDevices()` (*pyTempico.core.TempicoDevicesSearch* method), 3

G

`getAverageCycles()` (*pyTempico.core.TempicoChannel* method), 29
`getAverageCycles()` (*pyTempico.core.TempicoDevice* method), 9
`getBaudRate()` (*pyTempico.core.TempicoDevice* method), 9
`getDateTime()` (*pyTempico.core.TempicoDevice* method), 9
`getDatetime()` (*pyTempico.core.TempicoDevice* method), 9
`getDelay()` (*pyTempico.core.TempicoChannel* method), 29
`getDelay()` (*pyTempico.core.TempicoDevice* method), 10
`getFirmware()` (*pyTempico.core.TempicoDevice* method), 10
`getGeneratorFrequency()` (*pyTempico.core.TempicoDevice* method), 10
`getIdn()` (*pyTempico.core.TempicoDevice* method), 10
`getLastDelaySync()` (*pyTempico.core.TempicoDevice* method), 10
`getLastStart()` (*pyTempico.core.TempicoDevice* method), 11
`getLastSync()` (*pyTempico.core.TempicoDevice* method), 11
`getMaximumDateTime()` (*pyTempico.core.TempicoDevice* method), 11
`getMaximumDatetime()` (*pyTempico.core.TempicoDevice* method), 11
`getMinimumDateTime()` (*pyTempico.core.TempicoDevice* method), 12
`getMinimumDatetime()` (*pyTempico.core.TempicoDevice* method), 12
`getMode()` (*pyTempico.core.TempicoChannel* method), 29

- [getMode\(\)](#) (*pyTempico.core.TempicoDevice method*), [12](#)
[getModelIdn\(\)](#) (*pyTempico.core.TempicoDevice method*), [12](#)
[getNumberOfRuns\(\)](#) (*pyTempico.core.TempicoDevice method*), [13](#)
[getNumberOfStops\(\)](#) (*pyTempico.core.TempicoChannel method*), [30](#)
[getNumberOfStops\(\)](#) (*pyTempico.core.TempicoDevice method*), [13](#)
[getOverflowParameter\(\)](#) (*pyTempico.core.TempicoDevice method*), [13](#)
[getSerialNumber\(\)](#) (*pyTempico.core.TempicoDevice method*), [13](#)
[getSettings\(\)](#) (*pyTempico.core.TempicoDevice method*), [14](#)
[getStartEdge\(\)](#) (*pyTempico.core.TempicoChannel method*), [30](#)
[getStartEdge\(\)](#) (*pyTempico.core.TempicoDevice method*), [14](#)
[getStartSource\(\)](#) (*pyTempico.core.TempicoChannel method*), [30](#)
[getStartSource\(\)](#) (*pyTempico.core.TempicoDevice method*), [14](#)
[getState\(\)](#) (*pyTempico.core.TempicoChannel method*), [30](#)
[getState\(\)](#) (*pyTempico.core.TempicoDevice method*), [15](#)
[getStates\(\)](#) (*pyTempico.core.TempicoDevice method*), [15](#)
[getStatus\(\)](#) (*pyTempico.core.TempicoChannel method*), [31](#)
[getStatus\(\)](#) (*pyTempico.core.TempicoDevice method*), [15](#)
[getStopEdge\(\)](#) (*pyTempico.core.TempicoChannel method*), [31](#)
[getStopEdge\(\)](#) (*pyTempico.core.TempicoDevice method*), [16](#)
[getStopMask\(\)](#) (*pyTempico.core.TempicoChannel method*), [31](#)
[getStopMask\(\)](#) (*pyTempico.core.TempicoDevice method*), [16](#)
[getStopMaskMinimum\(\)](#) (*pyTempico.core.TempicoDevice method*), [17](#)
[getStopSource\(\)](#) (*pyTempico.core.TempicoChannel method*), [31](#)
[getStopSource\(\)](#) (*pyTempico.core.TempicoDevice method*), [17](#)
[getTempicoChannel\(\)](#) (*pyTempico.core.TempicoDevice method*), [17](#)
[getThresholdVoltage\(\)](#) (*pyTempico.core.TempicoDevice method*), [18](#)
[getVidPid\(\)](#) (*pyTempico.core.TempicoDevicesSearch method*), [3](#)
- I**
[id_tempico_channel](#) (*pyTempico.core.TempicoChannel attribute*), [27](#)
[id_tempico_device](#) (*pyTempico.core.TempicoChannel attribute*), [28](#)
[id_tempico_device](#) (*pyTempico.core.TempicoDevice attribute*), [4](#)
[idn](#) (*pyTempico.core.TempicoDevice attribute*), [5](#)
[incrementGeneratorFrequency\(\)](#) (*pyTempico.core.TempicoDevice method*), [18](#)
[incrementThresholdVoltage\(\)](#) (*pyTempico.core.TempicoDevice method*), [18](#)
[isEnabled\(\)](#) (*pyTempico.core.TempicoChannel method*), [32](#)
[isEnabled\(\)](#) (*pyTempico.core.TempicoDevice method*), [18](#)
[isIdle\(\)](#) (*pyTempico.core.TempicoDevice method*), [19](#)
[isIdleOrDisabled\(\)](#) (*pyTempico.core.TempicoDevice method*), [19](#)
[isOpen\(\)](#) (*pyTempico.core.TempicoDevice method*), [19](#)
[isPendingReadMessage\(\)](#) (*pyTempico.core.TempicoDevice method*), [19](#)
[isValidConsecutiveNumberListRows\(\)](#) (*pyTempico.core.TempicoDevice method*), [19](#)
[isValidNumberListRow\(\)](#) (*pyTempico.core.TempicoDevice method*), [20](#)
- M**
[measure\(\)](#) (*pyTempico.core.TempicoDevice method*), [20](#)
[mode](#) (*pyTempico.core.TempicoChannel attribute*), [28](#)
[module](#)
[pyTempico.core](#), [3](#)
- N**
[number_of_channels](#) (*pyTempico.core.TempicoDevice attribute*), [5](#)
[number_of_runs](#) (*pyTempico.core.TempicoDevice attribute*), [5](#)
[number_of_stops](#) (*pyTempico.core.TempicoChannel attribute*), [28](#)
- O**
[open\(\)](#) (*pyTempico.core.TempicoDevice method*), [21](#)
[openTempico\(\)](#) (*pyTempico.core.TempicoDevice method*), [21](#)
- P**
[parent_tempico_device](#) (*pyTempico.core.TempicoChannel attribute*), [28](#)
[port](#) (*pyTempico.core.TempicoDevice attribute*), [5](#)
[pyTempico.core](#)
[module](#), [3](#)

R

`readIdnFromDevice()` (*pyTempico.core.TempicoDevice method*), 21

`readMessage()` (*pyTempico.core.TempicoDevice method*), 21

`reset()` (*pyTempico.core.TempicoDevice method*), 21

S

`selfTest()` (*pyTempico.core.TempicoDevice method*), 22

`setAverageCycles()` (*pyTempico.core.TempicoChannel method*), 32

`setAverageCycles()` (*pyTempico.core.TempicoDevice method*), 22

`setDateTime()` (*pyTempico.core.TempicoDevice method*), 22

`setDatetime()` (*pyTempico.core.TempicoDevice method*), 22

`setGeneratorFrequency()` (*pyTempico.core.TempicoDevice method*), 22

`setMaximumDateTime()` (*pyTempico.core.TempicoDevice method*), 23

`setMaximumDatetime()` (*pyTempico.core.TempicoDevice method*), 23

`setMinimumDateTime()` (*pyTempico.core.TempicoDevice method*), 23

`setMinimumDatetime()` (*pyTempico.core.TempicoDevice method*), 23

`setMode()` (*pyTempico.core.TempicoChannel method*), 32

`setMode()` (*pyTempico.core.TempicoDevice method*), 23

`setNumberOfRuns()` (*pyTempico.core.TempicoDevice method*), 24

`setNumberOfStops()` (*pyTempico.core.TempicoChannel method*), 32

`setNumberOfStops()` (*pyTempico.core.TempicoDevice method*), 24

`setStartEdge()` (*pyTempico.core.TempicoChannel method*), 32

`setStartEdge()` (*pyTempico.core.TempicoDevice method*), 24

`setStartExternalSource()` (*pyTempico.core.TempicoChannel method*), 33

`setStartExternalSource()` (*pyTempico.core.TempicoDevice method*), 25

`setStartInternalSource()` (*pyTempico.core.TempicoChannel method*), 33

`setStartInternalSource()` (*pyTempico.core.TempicoDevice method*), 25

`setStopEdge()` (*pyTempico.core.TempicoChannel method*), 33

`setStopEdge()` (*pyTempico.core.TempicoDevice method*), 25

`setStopExternalSource()` (*pyTempico.core.TempicoChannel method*), 33

`setStopExternalSource()` (*pyTempico.core.TempicoDevice method*), 25

`setStopInternalSource()` (*pyTempico.core.TempicoChannel method*), 33

`setStopInternalSource()` (*pyTempico.core.TempicoDevice method*), 25

`setStopMask()` (*pyTempico.core.TempicoChannel method*), 33

`setStopMask()` (*pyTempico.core.TempicoDevice method*), 26

`setThresholdVoltage()` (*pyTempico.core.TempicoDevice method*), 26

`setThresholdVoltageToMaximum()` (*pyTempico.core.TempicoDevice method*), 26

`setThresholdVoltageToMinimum()` (*pyTempico.core.TempicoDevice method*), 27

`start_edge` (*pyTempico.core.TempicoChannel attribute*), 28

`stop_edge` (*pyTempico.core.TempicoChannel attribute*), 28

`stop_mask` (*pyTempico.core.TempicoChannel attribute*), 28

T

`TempicoChannel` (*class in pyTempico.core*), 27

`TempicoDevice` (*class in pyTempico.core*), 4

`TempicoDevicesSearch` (*class in pyTempico.core*), 3

`threshold` (*pyTempico.core.TempicoDevice attribute*), 5

`tryOpenDevices()` (*pyTempico.core.TempicoDevicesSearch method*), 4

V

`verifyPyTempico()` (*pyTempico.core.TempicoDevicesSearch method*), 4

W

`waitAndReadMessage()` (*pyTempico.core.TempicoDevice method*), 27

`writeMessage()` (*pyTempico.core.TempicoDevice method*), 27